



**COMSATS University Islamabad,
Sahiwal Campus.**

PRISVERDA

A Project Presented To

**COMSATS University, Islamabad
Sahiwal Campus**

**In partial fulfillment
of the requirement for the degree of**

Bachelor of Science in Computer Science (2021-2025)

By

Muhammad Ammar	CIIT/FA21-BCS-018/SWL
M.Muneeb Shahid	CIIT/FA21-BCS-019/SWL
M.Roman Zahid	CIIT/FA21-BCS-026/SWL

DECLARATION

We hereby declare that this system, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this website and accompanied report entirely based on our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Muhammad Ammar

M.Muneeb Shahid

M.Roman Zahid

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) “Prisverda” was developed by Muhammad Ammar (CIIT/FA21-BCS-018) a n d M.Muneeb Shahid (CIIT/FA21-BCS-019)and M.Roman Zahid (CIIT/FA21-BCS-026) under the supervision of “**Mr. Ali Sher Kashif** ” and co supervisor “**Ms. Madiha Fatima**” and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelor of Science in Computer Sciences.

Supervisor

Mr. Ali Sher Kashif

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr. Zafar Iqbal

Head of Department

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

The PRISVERDA project is an online e-commerce platform focused on delivering a seamless and efficient clothing shopping experience. It addresses the growing demand for digital retail solutions by creating a responsive, user-friendly website where users can browse, select, and purchase a wide variety of clothing products. With the rise of e-commerce and mobile shopping, PRISVERDA aims to provide a modern solution that is secure, scalable, and tailored to user needs.

The system includes essential functionalities such as user registration and login, product catalog management, shopping cart, payment gateway integration, and order tracking. Additionally, it emphasizes performance, user experience, and data security, ensuring that both customers and administrators can interact with the system efficiently.

The development of PRISVERDA is grounded in Agile methodology, which allows the project to adapt to changing requirements and incorporate continuous feedback. This iterative approach ensures the delivery of high-quality modules, rapid deployment of updates, and strong alignment with user expectations.

Relevance to academic course modules is clear, as the project draws upon principles of Software Engineering, Database Management, Web Technologies, and User Interface Design. It integrates both front-end and back-end development skills and mirrors the full software development lifecycle, from requirement analysis and design to implementation and testing. The project stands out for its real-world applicability and serves as a practical demonstration of key computer science concepts in action. PRISVERDA aims to be more than just a project. it aspires to be a scalable prototype for modern e-commerce applications in the fashion domain.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task. We are indebted to our project supervisor “**Mr. Ali Sher Kashif**”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work. And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Muhammad Ammar

M.Muneeb Shahid

M.Roman Zahid

Abbreviations

SRS	Software Requirement Specification
PC	Personal Computer
UI	User Interface
UX	User Experience
DB	Database
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
CRUD	Create, Read, Update, Delete
MVC	Model View Controller
API	Application Programming Interface
URL	Uniform Resource Locator

Table of Content

Chapter 1.....	1
Introduction	1
1. Introduction	2
1.1 Overview of the Project	2
1.2 The Motivation Behind Prisverda	2
1.3 Technology Stack and Its Benefits	2
1.4 Mobile-First Design and Responsive Experience.....	3
1.5 Key Features and Functional Goals	3
1.6 Development Methodology	3
1.7 Importance of COD for Target Markets.....	3
1.8 Alignment with Course Objectives	4
Chapter 2.....	5
Literature Review	5
2. Literature Review.....	6
2.1 Introduction to E-Commerce Evolution	6
2.2 User Experience (UX) and Interface Design.....	6
2.2.1 Importance of UX in E-Commerce	6
2.2.2 Responsive and Adaptive Design	6
2.3 The MERN Stack in E-Commerce Platforms	6
2.3.1 Adoption of the MERN Stack.....	6
2.3.2 Scalability and Real-World Implementation	7
2.4 Trust and Security in Online Transactions	7
2.4.1 Common Concerns in Digital Payments	7
2.4.2 Prisverda’s Focus on COD	7
2.5 Agile Methodology in Web Application Development	7
2.5.1 Why Agile for E-Commerce Projects	7
2.5.2 Benefits Realized in Prisverda.....	8
2.6 Market Gap in Regional E-Commerce	8
2.6.1 Lack of Localized Solutions	8
2.6.2 Prisverda vs. Traditional Platforms	8
Chapter 3.....	9
Methodology.....	9
3. Methodology.....	10
3.1 Introduction to Development Methodology.....	10
3.2 Why Agile Was Chosen	10
3.2.1 Flexibility in Changing Requirements.....	10
3.2.2 Continuous User Involvement.....	10
3.2.3 Early and Frequent Delivery	11
3.3 Agile Workflow Adopted	11
3.3.1 Sprint Planning	11
3.3.2 Daily Stand-Up Meetings	11
3.3.3 Sprint Execution	11

3.3.4 Sprint Review and Demo.....	11
3.3.5 Retrospective	12
3.4 Software Development Life Cycle (SDLC) Phases under Agile.....	12
3.4.1 Requirement Gathering and Analysis	12
3.4.2 System Design	12
3.4.3 Implementation.....	12
3.4.4 Testing	13
3.4.5 Deployment and Review	13
Chapter 4.....	14
Problem Statement.....	14
4.Problem Statement.....	15
4.1 Deliverable and Development Requirements	15
4.1.1 Deliverables	15
4.1.2 Technical Development Requirements	15
4.2 Current System	16
Chapter 5.....	18
Requirement Analysis.....	18
5. Requirement Analysis.....	19
5.1 Use Case Diagram(s).....	19
5.2 Use Case Descriptions	20
5.2.2 Get Registration Use Case Description:	22
5.2.3 View Products Use Case Description:	23
5.2.4 User Detail Use Case Description:	24
5.2.5 Check Prices Use Case Description:	25
5.2.6 Request for Order Use Case Description:	26
Chapter 6.....	28
Design and Architecture.....	28
6. Design and Architecture.....	29
6.1 System Architecture	29
6.2 Data Representation	29
6.3 Process Flow/Representation	31
Chapter 7.....	33
Design Models.....	33
7. Design Models.....	34
7.1 Sequence Diagram	34
7.2 Class Diagram	35
7.2.1 User Class:.....	35
7.2.2 Product Class:.....	36
7.2.3 Order Class:	37
7.2.4 Cart Class:	37
7.2.5 Payment Class:	38
Chapter 8.....	40
Implementation.....	40

8. Implementation.....	41
8.1 Algorithm.....	Error! Bookmark not defined.
8.2 External APIs	42
8.3 User Interface	43
8.4 Home Page	43
8.5 Services Page Interface design	44
8.6 Intuitive Product Categorization	45
8.7 Detailed Item Display Interface	46
8.8 Highlighted Features.....	47
8.9 Footer Section of the Website.....	48
8.10 Contact Us Page	49
8.11 User Login.....	50
Chapter 9.....	52
Testing and Evaluation.....	52
9. Testing and Evaluation.....	53
9.1 Manual Testing	54
9.2 System testing	55
9.3 Unit Testing.....	55
9.3.1 Unit Testing 1: User Login	55
9.3.2 Unit Testing 2: Edit Profile.....	55
9.4 Functional Testing.....	56
9.4.1 Functional Testing 1: Login with Different Roles.....	56
9.5 Integration Testing	56
9.6 Automated Testing	56
9.7 Tools Used	56
Chapter 10.....	58
Conclusion and Future work	58
10. Conclusion and Future Work.....	59
10.1 Conclusion	59
10.2 Future Work	60
10.2.1 Payment Gateway Integration	60
10.2.2 Real-Time Order Tracking System.....	60
10.2.3 Personalized Recommendation Engine	61
10.2.4 Email and SMS Notifications	61
10.2.5 Mobile Application Development	61
10.2.6 Multilingual and Multi-Currency Support.....	62
Conclusion of Future Work	62
11. References	

List of Figures

Figure 4.2 Current System Workflow Of Prisverda.....	17
Figure 5. 1 Use Case Diagram For Prisverda E-Commerce Platform	20
Figure 6.1 System Architecture Diagram Of Prisverda E-Commerce Website	29
Figure 6.2 Entity Relationship Diagram (Erd) For The Prisverda	30
Figure 6.4 User Order Process Flow For The Prisverda Clothing Store	32
Figure 7.1 Sequence Diagram of Prisverda	35
Figure 7.2 Class Diagram – System Structure	39
Figure 8.1 Home Screen Of Prisverda	44
Figure 8.2 Service Page Interface Design	45
Figure 8.3 Intuitive Product Categorization Of Prisverda	46
Figure 8.4 Detailed Item Display Interface Of Prisverda Item	47
Figure 8.5 Highlighted Features Interface Design Of Prisverda	48
Figure 8.6 Footer Section Of The Prisverda	49
Figure 8.7 Contact Us Page Interface Of Prisverda	50
Figure 8.8 User Login Interface Of Prisverda	51

List of Tables

Table 4.1: Technologies And Tools Used In The Development Of Prisverda	16
Table 5.1 Login Use Case Details For "Prisverda" E-Commerce Platform	21
Table 5.2 Get Registration Use Case Details For “Prisverda” E-Commerce Platform ..	22
Table 5.3 View Products Use Case Details For “Prisverda” E-Commerce Platform	23
Table 5.4 Use Case Details For “Prisverda” E-Commerce Platform	25
Table 5.5 Check Prices Use Case Details For “Prisverda” E-Commerce Platform	26
Table 5.6 Request For Order Use Case Details For “Prisverda”	27
Table 8.1 Details Of Apis Used In The Project	42
Table 9.1 To Verify That The Login Form Functions Correctly	55
Table 9.2 Ensure That The Edit Profile Form Works Correctly	55
Table 9.3 To Confirm That Users Are Redirected To The Correct Dashboard	56
Table 9.4 Integration Testing Results For Core Functionalities	56
Table 9.5 Automated Testing Results Using Selected Tools	57

Chapter 1

Introduction

1. Introduction

1.1 Overview of the Project

Prisverda is a full-stack e-commerce web application tailored for the online clothing industry. It is designed to deliver a **seamless, secure, and intuitive digital shopping experience**, particularly for consumers in regions where online shopping is still maturing. Developed using the **MongoDB, React.js**, and **Node.js**. Prisverda is engineered to meet both user expectations and technical demands, offering a scalable and modular architecture that supports rapid updates and efficient feature integration.

1.2 The Motivation Behind Prisverda

The digital retail sector has grown exponentially in recent years, especially within fashion and apparel. However, gaps still exist in emerging markets where users demand greater usability, payment flexibility, and trust before adopting e-commerce fully. **Prisverda was motivated by three key insights:**

- Many regional customers still hesitate to use online payment methods due to concerns about fraud or lack of digital literacy.
- Local retailers lack effective digital storefronts with streamlined product navigation, secure login systems, and organized catalogs.
- There is increasing demand for a **mobile-first experience** that simplifies the process of discovering, selecting, and purchasing fashion items.

Prisverda addresses these gaps by focusing on **Cash on Delivery (COD)** support, **responsive design**, and a **clean, intuitive interface** that meets modern user expectations while staying accessible to all demographics.

1.3 Technology Stack and Its Benefits

Prisverda is built using a popular full-stack JavaScript solution:

- **MongoDB:** A NoSQL database that allows flexible schema design for dynamic product catalogs.
- **React.js:** A frontend library for building interactive UIs using component-based architecture, enabling efficient rendering and scalability.
- **Node.js:** A powerful backend runtime that allows handling of concurrent requests, making it ideal for high-traffic platforms.

By leveraging MERN, the development process remains consistent and efficient since the entire stack is based on JavaScript, reducing context-switching and promoting faster debugging and collaboration.

1.4 Mobile-First Design and Responsive Experience

Understanding the shift toward mobile commerce (m-commerce), Prisverda follows a **mobile-first design approach**. This ensures that users accessing the platform via smartphones or tablets get a full-featured experience without lag or layout issues. Features like **touch-friendly buttons**, **scalable images**, and **responsive grid layouts** were incorporated from the beginning using **Material-UI**, **Bootstrap**, and CSS3 media queries.

This approach improves user retention, as research shows that over 60% of e-commerce traffic in Pakistan comes from mobile devices. Prisverda ensures that customers browsing on-the-go can quickly find products, add them to their cart, and place orders with minimal steps.

1.5 Key Features and Functional Goals

The project aims to offer an all-in-one e-commerce platform for clothing, fulfilling both **user-facing** and **admin-facing** needs. Some of the key features include:

- **Secure Login/Registration System**

Using encrypted authentication and protected endpoints, ensuring secure access to accounts.

- **Product Browsing with Search and Filters**

Users can view detailed descriptions, prices, and images, and refine results using filters such as price and category.

- **Shopping Cart & Checkout with COD**

Users can manage their cart and complete their purchase through a streamlined COD checkout process.

1.6 Development Methodology

Prisverda adopted an **Agile development methodology**, structured around **iterative sprints**. This allowed:

- Early delivery of working modules (e.g. login system, cart)
- Rapid adaptation to stakeholder feedback
- Incremental improvements in UI/UX and performance
- Better risk mitigation and flexibility in planning

Each sprint consisted of requirement analysis, design, implementation, testing, and documentation aligning with academic expectations of a full SDLC application.

1.7 Importance of COD for Target Markets

In countries like Pakistan, where digital payment adoption is still growing, **Cash on Delivery remains the preferred mode of payment**, with studies showing more than **90% of online transactions** processed

through COD. Prisverda integrates this payment model seamlessly, building **customer trust** and ensuring greater completion rates for purchases.

Unlike online payment systems which may require third-party gateways and introduce complexity, COD is:

- Simple for users
- Reduces friction during checkout
- Increases conversion rates by avoiding upfront commitments

COD is not only a technical feature, but also a **market-aligned strategy** to improve adoption and engagement.

1.8 Alignment with Course Objectives

Prisverda directly integrates knowledge from various **computer science course modules**:

- **Software Engineering:** Requirements gathering, Agile workflows, UML diagrams, and SDLC phases
- **Web Development:** HTML, CSS, React, RESTful APIs
- **Database Management:** NoSQL modeling with MongoDB, CRUD operations, and schema flexibility
- **Human-Computer Interaction:** Emphasis on usability, accessibility, and responsive design
- **Cybersecurity Basics:** Secure login logic, password hashing, and data protection strategies

Chapter 2

Literature Review

2. Literature Review

2.1 Introduction to E-Commerce Evolution

E-commerce has transformed global commerce by enabling businesses to sell products online, eliminating physical barriers. The clothing and fashion segment is particularly competitive, requiring platforms to provide smooth and engaging customer experience. As online shopping grows, customer expectations have also evolved. They now demand faster load times, mobile compatibility, secure transactions, and recommendations. Research has shown that e-commerce success is tightly linked to technology, user trust, and design.

Prisverda draws on existing research and industry practices to develop a solution tailored for the Pakistani market, with emphasis on Cash on Delivery, user experience, and technical scalability.

2.2 User Experience (UX) and Interface Design

2.2.1 Importance of UX in E-Commerce

A 2021 study published on *ResearchGate* highlights that users are more likely to trust and revisit platforms that provide clean, simple, and intuitive interfaces. In fact, the study concluded that improving a website's layout and responsiveness raised user satisfaction from 23% to over 54%.

2.2.2 Responsive and Adaptive Design

Modern users access shopping platforms from various devices. Responsive design ensures that websites **adjust layouts automatically** across desktops, tablets, and smartphones. Websites that fail to deliver a smooth mobile experience suffer from **high bounce rates** and **low conversions**.

Prisverda addresses this by:

- Using **Material-UI** for grid layouts and component styling.
- Implementing **CSS media queries** to ensure consistency across devices.
- Building reusable **React components** to manage dynamic rendering.

2.3 The MERN Stack in E-Commerce Platforms

2.3.1 Adoption of the MERN Stack

According to a paper published on *JETIR.org*, the MERN stack offers a powerful toolkit for developing fast and scalable e-commerce solutions. The stack consists of:

- **MongoDB** – Flexible NoSQL database storing JSON-like documents.
- **React.js** – Dynamic frontend library for building user interfaces.
- **Node.js** – Backend JavaScript runtime supporting asynchronous request handling.

This technology stack allows end-to-end JavaScript development, leading to better code sharing, simpler debugging, and faster deployment.

2.3.2 Scalability and Real-World Implementation

Companies such as Flipkart and Myntra have adopted stack-based microservice architectures like MERN to handle millions of users. Features like asynchronous non-blocking I/O in Node.js and the document-oriented approach in MongoDB allow Prisverda to:

- Scale horizontally with increased traffic.
- Deliver real-time data updates (e.g. cart status).
- Handle high concurrency on the server.

2.4 Trust and Security in Online Transactions

2.4.1 Common Concerns in Digital Payments

One of the most cited reasons users avoid online shopping is **lack of trust in online payments**. Concerning fraud, phishing, and data leaks make users reluctant to enter card or bank details online. These fears are **amplified in developing countries**, where regulatory protections are either weak or poorly enforced.

2.4.2 Prisverda's Focus on COD

According to *trade.gov*, over 90% of online shoppers in Pakistan choose **Cash on Delivery (COD)** due to trust concerns. Another study by *Dyl-Ventures.com* confirmed that 79% of shoppers feel more comfortable paying upon product arrival.

Prisverda integrates COD into:

- Increase **conversion rates** by removing the barrier of upfront payment.
- Build **customer trust** through transparency.
- Serve customers without access to digital payment methods.

By aligning with local payment habits, Prisverda not only encourages new users but also minimizes cart abandonment due to payment anxiety.

2.5 Agile Methodology in Web Application Development

2.5.1 Why Agile for E-Commerce Projects

Agile methodology promotes:

- Incremental development
- Fast iterations

- Continuous user feedback

According to *ccaps.umn.edu* and *brainspate.com*, Agile is ideal for web development due to the **fluid nature of requirements**. In retail, user preferences change frequently Agile's flexibility allows for adapting scope and features throughout development.

2.5.2 Benefits Realized in Prisverda

By adopting Agile, Prisverda:

- Delivered early prototypes (e.g., user login, catalog) for testing and validation.
- Received feedback from users and supervisors to improve mid-cycle features.
- Maintained high productivity and transparency through daily stand-ups and sprint reviews.

For example, after early feedback, the team improved cart behavior, added a price filter, and redesigned the homepage layout all within one sprint. Agile's emphasis on working software over documentation also ensured a **functioning system was always available** for testing and demonstration.

2.6 Market Gap in Regional E-Commerce

2.6.1 Lack of Localized Solutions

Many retailers still operate manually or use generic e-commerce templates that don't address local needs.

Problems include:

- English-only interfaces
- Non-functional payment gateways
- Slow mobile performance
- No COD or local delivery integrations

Prisverda is **built** with future provisions for:

- English language support
- Easy to read
- Location-based product visibility

2.6.2 Prisverda vs. Traditional Platforms

Whereas many platforms focus on premium fashion or international audiences, Prisverda targets **local affordability, familiarity (COD), and ease of use**, especially for first-time online shoppers.

Chapter 3

Methodology

3. Methodology

3.1 Introduction to Development Methodology

Software development methodologies define the structure and workflow of a project's lifecycle. For a dynamic and user-centric project like Prisverda where continuous refinement, stakeholder feedback, and changing requirements were expected a **flexible and iterative methodology** was necessary. After evaluating various approaches, the **Agile Software Development Life Cycle (Agile SDLC)** was selected.

Agile focuses on **iterative progress**, **working software delivery**, and **cross-functional collaboration**. It aligns with Prisverda's objective to adapt to feedback quickly, deliver incremental progress, and maintain high user involvement throughout development.

3.2 Why Agile Was Chosen

3.2.1 Flexibility in Changing Requirements

In traditional models like Waterfall, once requirements are fixed, any changes are costly. However, in Prisverda's case:

- User expectations evolved during development.
- New ideas emerged during testing (e.g. adding sorting filters, improving cart persistence).
- Features had to be prioritized based on feedback.

Agile allowed the team to embrace these changes without disrupting the entire system. Features could be added, dropped, or re-prioritized between sprints.

3.2.2 Continuous User Involvement

Agile encourages constant interaction with stakeholders in this case, the project supervisor, team members, and potential users. With **frequent sprint reviews** and **demos**, feedback was received early and often, allowing refinement at each stage. For instance, after the first demo, the team received suggestions to:

- Improve product layout on mobile.
- Add a banner for promotions.
- Refactor the checkout process to require fewer steps.

This feedback was incorporated in the next sprint.

3.2.3 Early and Frequent Delivery

By dividing the work into short **sprints** (2–3 weeks), the team ensured that each module — login, product catalog, cart, and order — was developed, tested, and delivered incrementally. This made troubleshooting easier and allowed parts of the system to be used for early validation.

3.3 Agile Workflow Adopted

3.3.1 Sprint Planning

Each sprint began with a **planning meeting** where goals, features, and tasks were discussed. The backlog was broken into smaller tasks (user stories), such as:

- “As a user, I want to register and log in securely.”
- “As a customer, I want to sort products by price.”

Tasks were estimated and assigned based on priority and team capacity.

3.3.2 Daily Stand-Up Meetings

Every day, the team conducted short stand-up meetings to discuss:

- What was done yesterday
- What will be done today
- Any blockers or issues

This promoted **team accountability**, quick problem-solving, and transparent progress tracking.

3.3.3 Sprint Execution

During the sprint, each member focused on specific features — for example:

- Frontend developer built the product list and filters using React.
- Backend developer created REST APIs for cart and orders using Express and Node.
- Database schemas were designed in MongoDB and linked via Mongoose models.

Progress was tracked using tools like Trello or GitHub Projects, where tasks were moved across boards (ToDo → In Progress → Done).

3.3.4 Sprint Review and Demo

At the end of the sprint, a working version of the system was demonstrated to the supervisor and internal team. This included showcasing:

- Functional modules (e.g., login working with JWT).
- Any UI improvements.
- Backend response logs via Postman or browser.

Feedback was collected for future improvements.

3.3.5 Retrospective

Each sprint closed with a **retrospective meeting**, where the team discussed:

- What went well?
- What didn't go as planned?
- What can be improved in the next sprint?

This led to meaningful improvements. For example:

- After one sprint, the team adopted better naming conventions for backend routes.
- After another, they started including mobile-first testing as a checklist item.

3.4 Software Development Life Cycle (SDLC) Phases under Agile

3.4.1 Requirement Gathering and Analysis

Initial requirements were collected from stakeholder interviews and research on local e-commerce practices. These were organized as **user stories** in the product backlog. Additional requirements (e.g. filter by category, Google Maps integration) were added based on evolving needs.

3.4.2 System Design

Wireframes and UI mockups were created in **Figma**, showcasing the layout of pages like the homepage, product grid, login, cart, and admin panel. Based on these mockups, component trees were drawn for the React frontend. The backend API structure and MongoDB document schemas were also designed here.

3.4.3 Implementation

Development was made in incremental builds:

- React components were developed with reusable logic.
- Node.js handled REST API routing and middleware logic.
- MongoDB models were connected via Mongoose and tested using Postman.

Each module was implemented and pushed to a version control system.

3.4.4 Testing

Manual and unit testing were done during and after development. Integration testing ensured that modules interacted correctly (e.g., product API → cart → checkout → order record creation). Automated tests using tools like **Jest** were created for specific backend functions.

3.4.5 Deployment and Review

The complete system was hosted locally for review, and optionally deployed using services like **Heroku**, **Vercel**, or **Netlify** (for frontend). A build folder was generated for production testing. Final presentation and documentation were based on the latest stable build.

Chapter 4

Problem Statement

4.Problem Statement

In the current digital era, traditional clothing retail is rapidly transitioning to online platforms. However, many existing e-commerce clothing websites suffer from issues such as poor user experience, limited functionality, and inadequate security in transaction handling. Customers often face challenges in browsing products efficiently, receiving accurate product and completing secure purchases.

There is a need for a robust, user-friendly, and responsive online clothing store that addresses these shortcomings by offering a seamless shopping experience, secure transactions, and efficient product management. The system should also facilitate administrative control over inventory, orders, and user management while ensuring a scalable infrastructure that can adapt to future growth and changing user needs.

PRISVERDA aims to solve these problems by providing a well-designed, secure, and intuitive e-commerce platform tailored specifically for retail clothing.

4.1 Deliverable and Development Requirements

4.1.1 Deliverables

- Fully functional e-commerce website
- Secure customer login & registration system
- Shopping cart and checkout system
- Contact page with Google Maps location

4.1.2 Technical Development Requirements

The following table, Table 4.1 presents the technologies and tools used in the development of the Prisverda ecommerce platform. This style of table numbering and captioning will be followed consistently throughout the report.

These tools and technologies collectively enabled the full-stack development of Prisverda under the MERN architecture (MongoDB, Express.js, React.js, Node.js). They not only provided functional coverage but also ensured ease of maintenance, code reusability, and integration of third-party services like payment modules and Google Maps.

Table 4.1: Technologies and Tools Used in the Development of Prisverda

Tools And Technologies	Tools	Version	Rationale
	MS Visual Studio	2015	Web based project
	MongoDB	2024	DBMS
	MS Project	2016	Gantt Chart
	Adobe illustrator	CC	Design Work
	MS Word	2015	Documentation
	MS Power Point	2015	Presentation
	Figma	2.0.5	Mockups Creation
	Technology	Version	Rationale
	Html	5	Programming language
	Material UI	7	React component library
	CSS	3	Programming language
	Bootstrap	5	Programming language
	Java script	11	Programming language
	React js	18.2	Frontend framework
	Node js	11.5	Backend framework
	Express	14.9.2	Backend library

4.2 Current System

The current system is entirely offline and manually managed. Customers visit the physical store to browse products, place orders, and make payments. Inventory and customer data are maintained using spreadsheets and paper records. There is limited reach beyond the local customer base, and the absence of an e-commerce platform restricts growth.

The following figure, **Figure 4.1**, represents the **Current System Workflow of Prisverda**, providing an overview of the existing manual and offline processes, including in-store customer visits, product browsing, order placement, and inventory management using paper records and spreadsheets.

In contrast, a digitized system would streamline this workflow by automating each stage—from product browsing to checkout, and inventory management enhancing both customer convenience and operational control.

Thus, the current system workflow, though functional at a basic level, is not scalable and inhibits growth, making it imperative for Prisverda to transition to a comprehensive e-commerce platform.

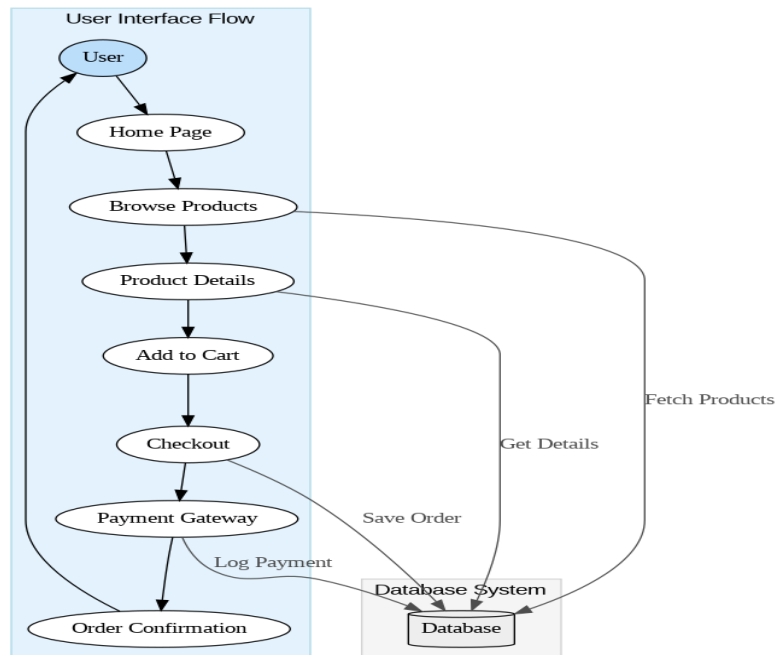


Figure 4.1 Current System Workflow of Prisverda

Chapter 5

Requirement Analysis

5. Requirement Analysis

Requirement Analysis is a crucial phase in system development, where the goal is to understand the needs and expectations of the stakeholders and to translate them into a clear set of system requirements. For the **Prisverda** e-commerce platform, this process involves analyzing both the functional and non-functional requirements, as well as identifying the key user interactions through **use case diagrams**.

5.1 Use Case Diagram(s)

A **Use Case Diagram** serves as a high-level visual representation of how users (or actors) interact with the system. For the e-commerce platform **Prisverda**, the diagram includes two primary actors: **Customer** and **Admin**. These actors interact with various system functionalities to perform key tasks.

- **Customer:** A Customer can browse products, add them to the cart, place orders, and like them to purchase later. They can also register, log in, and manage their profile.
- **Admin:** An Admin can manage product listings, process orders, manage customer accounts, and oversee system operations.

These interactions are represented through various **use cases** such as **Login, View Products, Add to Cart, Place Order**, and more.

The following **figure 5.1** illustrates the use case diagram for the "**Prisverda**" e-commerce platform.

It visualizes the interaction between two primary actors: the **Customer** and the **Admin**. The diagram highlights the key functionalities available to each actor.

The use case diagram for Prisverda effectively captures the core functionalities of an e-commerce platform by modeling the interactions between Admin and Customer actors and the system. It supports standard e-commerce operations including user registration, product interaction, shopping cart usage, and order processing. However, role clarity (e.g., who can "Create Product") could be improved for greater consistency with typical e-commerce design

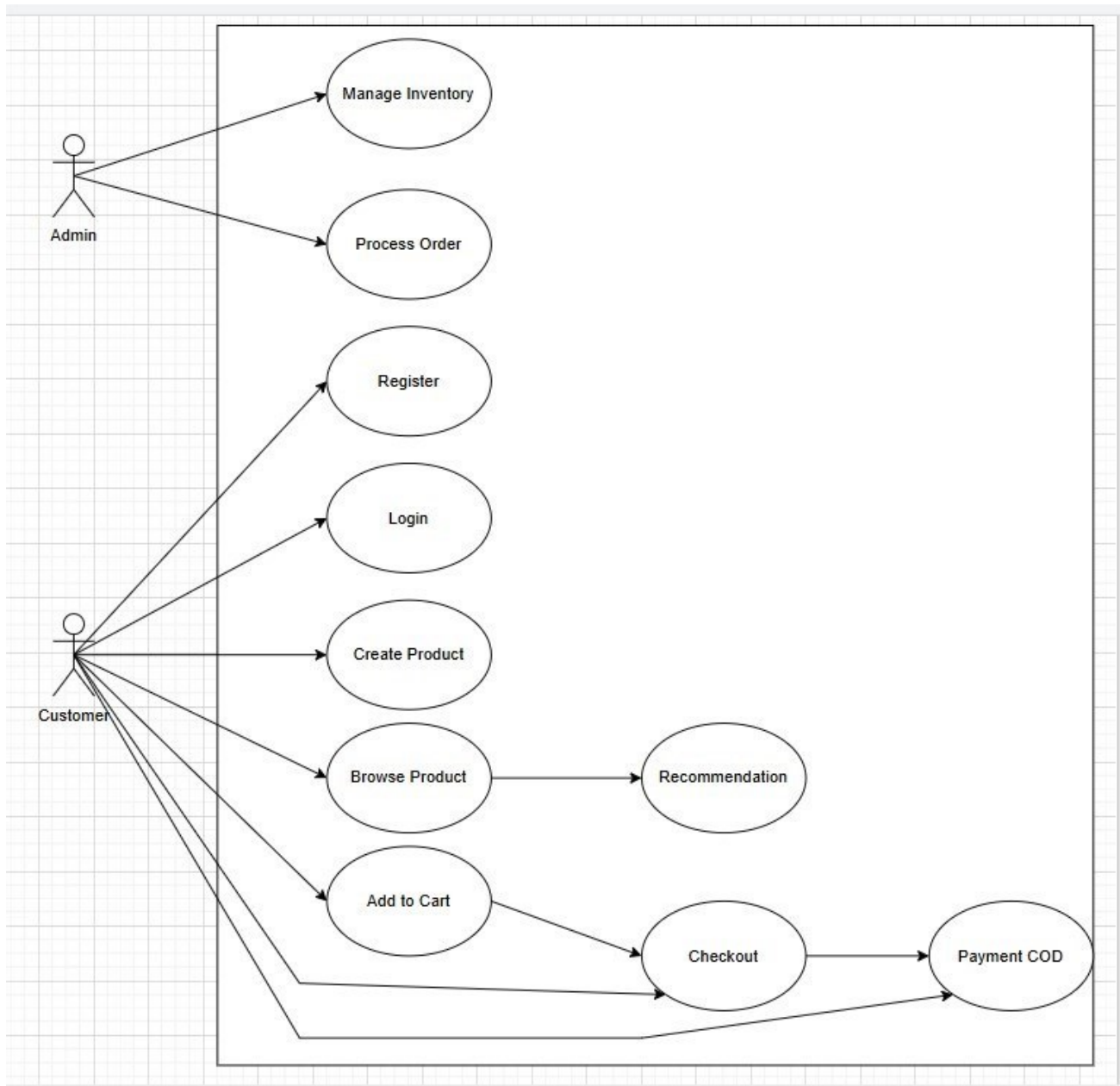


Figure 5.1 Use Case Diagram for Prisverda E-Commerce Platform

5.2 Use Case Descriptions

Below is a table describing the key use cases identified for the **Prisverda** platform. Each use case includes the involved actors, description, and the flow of events.

The following table illustrates the use case scenarios for the "Prisverda" e-commerce platform.

1. Customer Use Case Description:

The Customer Use Cases for the Prisverda e-commerce platform encompass a wide array of interactive features that empower users to seamlessly navigate the website and carry out essential shopping activities. These use cases are meticulously designed to ensure a user-friendly, engaging, and personalized shopping experience, catering to both new and returning customers. At the core,

the platform allows users to browse a diverse range of product categories, each containing detailed product pages with specifications, pricing, availability, size options, and high-resolution images. Customers can also benefit from intelligent product recommendations tailored to their browsing history, preferences, and previous purchases, enhancing product discovery and promoting relevant items.

In addition to product exploration, users can add desired items to a virtual shopping cart, modify quantities, or remove products as needed. The cart is dynamically updated to reflect the latest selections and total costs, including any applicable discounts or promotions. Once satisfied with their selections, customers can proceed to a secure and streamlined checkout process where they can choose payment methods, enter shipping details, and confirm their order. Payment options typically include credit/debit cards, online wallets, or cash on delivery, offering flexibility to the customer.

Beyond shopping functionalities, registered users can also manage their account settings, including updating personal information, changing passwords, viewing order history, tracking current orders, and managing saved addresses. These features are all backed by a responsive support system, including FAQs and direct contact options, ensuring that users have access to help whenever needed. Collectively, these customer use cases form the backbone of the Prisverda platform's interactive experience, ensuring efficiency, reliability, and satisfaction throughout the shopping journey.

Table 5.1 Login Use Case Details for "Prisverda" E-Commerce Platform

Use case id	Login_01
Use case name	Login
Actors	User
Description	A user of the System logs into the System.
Trigger	User requests to log in.
Pre-condition	The user has an account.
Post-condition	The actor is now logged into the system.
Normal flow	This use case starts when an actor wants to login into the system. The system requests that the actor enter e-mail and password. The actor enters e-mail and password. The system validates the entered e-mail and password and login into the system.

Alternative flow	For example, a Valid e-mail or an Invalid Password If in the basic flow the system cannot find the valid e-mail or the password is invalid, an error message is displayed.
Exceptions	Incorrect credentials. Request to re-enter the login details.
Business rules	The system must be connected to the network.

5.2.2 Get Registration Use Case Description:

The **Get Registration Use Case** involves the essential process of creating a new user account on the Prisverda e-commerce platform. This use case is initiated when a customer or admin selects the registration link, typically located on the login page or website header. Upon selecting this link, the user is directed to the registration page, where they are required to provide specific information such as their name, email address, password, and other relevant personal details.

The system verifies the provided data to ensure it meets the required format and does not duplicate existing user records. Once the information is validated, the system stores the user's details in the database, associating them with a unique user ID for future identification and login purposes. Successful registration grants the user access to various platform features, including browsing products, placing orders, and managing account settings.

If any mandatory fields are left incomplete or if the email address is already registered, the system prompts the user with appropriate error messages, guiding them to correct the information. Additionally, the registration process may include security measures such as email verification or CAPTCHA to prevent unauthorized access or spam registrations.

Table 5.2 Get Registration Use Case Details for “Prisverda” E-Commerce Platform

Use case id	Registration_02
Use case name	Get Registration
Actors	Customer
Description	A user of the System creates an account.
Trigger	Select the register link

Pre-condition	The user is not currently registered in the system and has accessed the registration page via the provided registration link.
Postcondition	The user's details are stored in the database.

5.2.3 View Products Use Case Description:

The **View Products Use Case** outlines the process through which customers can explore and interact with the product listings on the Prisverda e-commerce platform. This use case is initiated once the user successfully logs into the system, granting them access to the comprehensive product catalog. Upon login, the customer is presented with a range of product categories, each showcasing available items with relevant filters and sorting options, such as price, brand, popularity, and new arrivals.

The system displays detailed information for each product, including product name, high-resolution images, price, availability status, product specifications, size and color options, and customer reviews or ratings. This allows users to make well-informed purchasing decisions based on their preferences and needs.

Customers can navigate through the product listings, click on specific products to view more indepth details, and access recommended or related products. Additionally, they can refine their search by applying filters, sorting by price range, or searching for specific keywords to locate desired items quickly.

Once a product is selected, the user can proceed with further actions such as adding the item to the cart, saving it to a wishlist, or directly proceeding to checkout for immediate purchase. If the user is not logged in or if there is a system error (e.g., server downtime), the platform may restrict access to specific product details or actions, prompting the user to either log in or try again later.

Table 5.3 View Products Use Case Details for "Prisverda" E-Commerce Platform

Use case id	View Products_03
Use case name	View Products
Actors	Customer
Description	Customers can view all the available products and select the product of his/her own choice.
Trigger	Users want to request to see the product details.
Pre-condition	The actors log in to the system

Postcondition	The customer sees the information related to a particular product and selects the product of his/her own choice.
Normal flow	The users log in to the system. View the products
Alternative flow	The user did not register his /herself and did not view the available products details.
Exceptions	The error occurs in the Website. Server is down
Business rules	User registration is required.

5.2.4 User Detail Use Case Description:

The **Enter Details Use Case** involves the critical step of gathering necessary customer information after a registered user initiates a product purchase on the Prisverda e-commerce platform. This use case is triggered once a customer has successfully logged in and selected a product for purchase. Before proceeding to checkout, the customer is required to provide essential information, including personal details such as name, email address, contact number, and delivery address.

Additionally, the system may prompt the customer to enter specific information relevant to the order, such as preferred delivery method or any special instructions for the delivery. If the product being purchased is associated with specific requirements (e.g., vehicle-related products that require vehicle details), the customer must provide such information as well to ensure accurate order processing.

Once all required fields are completed, the system validates the entered information for completeness and accuracy. Any missing or incorrectly formatted data prompts an error message, requesting the customer to review and correct the information before proceeding. After successful validation, the system stores the customer's information and generates a confirmation message, indicating that the details have been successfully submitted and are ready for order processing.

In cases where the admin does not accept the request or if the server is down, the system notifies the customer with an appropriate error message. Similarly, if there is a network or connectivity issue, the system advises the user to retry later.

Table 5.4 Use Case Details for "Prisverda" E-Commerce Platform

Use case id	Details_04
Use case name	Enter details
Actors	Customer
Description	The customer must enter all the necessary information which includes personal as well as vehicle details.
Trigger	Customers want to buy a product.
Precondition	Customers must get registered on the Website.
Postcondition	A confirmation message is received by the customer. Customers fill out their personal information forms.
Normal flow	The customer successfully logs in to the system. Customers successfully get registered.
Alternative flow	Admin does not accept the request.
Exceptions	There is no internet. Server down

5.2.5 Check Prices Use Case Description:

Customers can access detailed price information through various sections of the platform, including product listings, product detail pages, and promotional banners. Each product entry provides essential pricing details, such as the base price, applicable discounts, and final sale price. Additionally, if multiple sizes or variations of a product are available, the system displays corresponding prices for each option.

The platform also offers sorting and filtering options, enabling users to organize products by price range, from low to high or vice versa. This functionality is particularly useful for budget-conscious customers seeking products within a specific price bracket. Furthermore, customers can view special offers, promotional discounts, and seasonal sales to make more cost-effective purchasing decisions.

If the customer is not logged in, the system may restrict access to detailed pricing information, prompting them to either log in or register to proceed. In cases of system errors, such as server downtime or network issues, the platform may be unable to display pricing data accurately, and an error message will be presented to the user.

Table 5.5 Check Prices Use Case Details for "Prisverda" E-Commerce Platform

Use case id	Check prices_05
Use case name	Check prices
Actors	Customer
Description	Customers can view the prices.
Trigger	Users want to request to see the check prices.
Pre-condition	The actors log in to the system.
Postcondition	The customer sees all available prices.
Normal flow	The users log in to the system.
Alternative flow	The user has not registered his/herself and has not viewed the available prices.
Exceptions	The error occurs in the application. The server is down.
Business rules	User registration is required.

5.2.6 Request for Order Use Case Description:

The **Request for Order Use Case** outlines the process in which a registered customer places an order for a selected product on the Prisverda e-commerce platform. This use case is initiated after the customer has successfully logged in, browsed the product listings, and added desired items to the shopping cart. Upon confirming the selection, the customer proceeds to request the order by initiating the checkout process.

During the checkout process, the system prompts the customer to review the order summary, including product details, quantities, prices, and any applicable discounts. The customer is then required to fill out personal information forms, such as shipping address, contact information, and payment method. This step ensures that the system collects all necessary data to accurately process and deliver the order.

Once the customer submits the order request, the system verifies the availability of the selected items and confirms the payment method. If the order is successfully processed, the system generates a confirmation message that is displayed to the customer and sent to their registered email address.

The confirmation includes the order number, estimated delivery date, and a detailed summary of the purchased items.

In case of exceptions, such as product unavailability or admin rejection of the order request, the system notifies the customer with a relevant error message and provides options to modify the order or contact customer support. Additionally, if the system experiences technical issues, such as a server outage or internet disruption, the order request may be delayed or temporarily unavailable, prompting the customer to retry later.

Table 5.6 Request for Order Use Case Details for "Prisverda" E-Commerce Platform

Use case id	Request for order_06
Use case name	Request for order
Actors	Customer
Description	After selecting the product user orders it
Trigger	Customers want to order the product.
Precondition	The customer must be registered and place the order.
Postcondition	A confirmation message is received by the customer. Customers fill out their personal information forms.
Normal flow	The customer successfully logs in to the system. Customers successfully order the product.
Alternative flow	Admin does not accept the request. No requested product is available.
Exceptions	There is no internet. Server down
Business rules	The request is rejected when the customer does not sign the int.

Chapter 6

Design and Architecture

6. Design and Architecture

The design and architecture of Prisverda are structured around a modular, scalable, and secure client-server model using the MERN stack—MongoDB, Express.js, React.js, and Node.js. The frontend, built with React, utilizes reusable functional components and integrates Bootstrap and Material-UI to ensure responsiveness and consistency across devices. React Router enables seamless navigation, while the Context API manages global state for user authentication and cart data. The backend, developed with Node.js and Express, provides RESTful APIs for user actions, product management, cart operations, and order processing. MongoDB stores application data in a flexible, document-based schema using Mongoose, with collections for users, products, carts, and orders. Key backend features include secure password hashing with bcrypt, role-based access control for admin routes, and token-based authentication (JWT-ready). The system enforces modular separation between UI, logic, and data layers, supporting future enhancements such as microservices. Security is prioritized through HTTPS, input validation, and protected API endpoints. External tools like Google Maps API enhance the user experience on the Contact page. Overall, the architecture supports clean UI interaction, real-time cart and order updates, secure transactions, and future scalability, making Prisverda a robust and extensible platform.

6.1 System Architecture

The structure of the system explains the working of the system and describes the components of the project, their relationships, and how they interact with each other and with the architecture. As Shown in Figure 6.1.

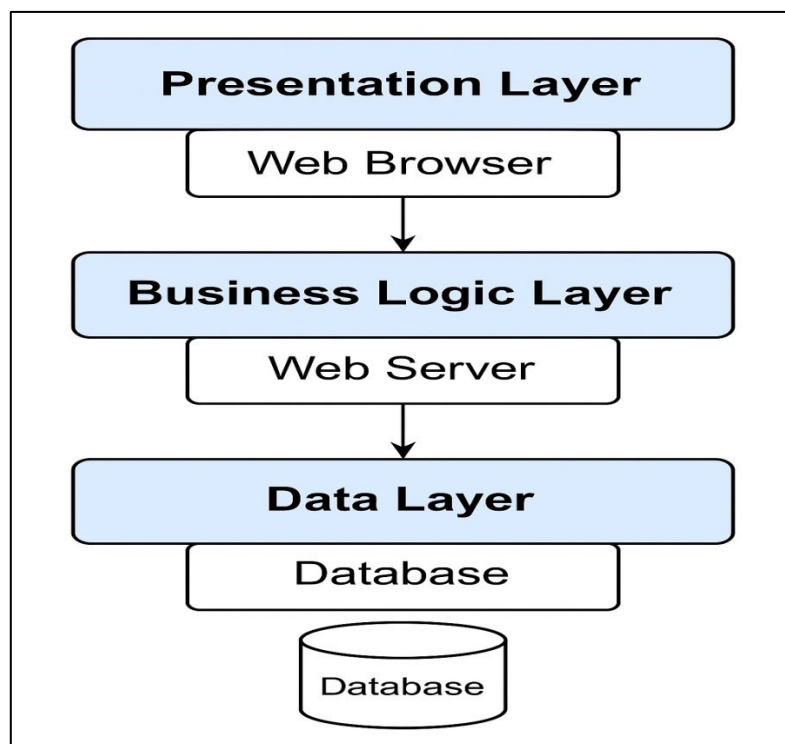


Figure 6.1 System Architecture Diagram of PRISVERDA E-Commerce Website

6.2 Data Representation

Data representation in Prisverda refers to how the application **stores, retrieves, and manipulates data** to support all functional requirements. The system uses **MongoDB**, a NoSQL

database, to store information as **JSON-like documents**. Unlike relational databases, MongoDB provides a flexible schema, allowing efficient data modeling for dynamic structures like shopping carts, orders, and product listings.

The backend uses **Mongoose**, an Object Data Modeling (ODM) library for MongoDB, which enables schema enforcement, validation, and easy integration with Node.js applications.

The main entities include:

- **User:** Stores user-related information such as user ID, name, email, password, and address.
- **Product:** Contains product-specific details like product ID, name, description, price, image, and category.
- **Order:** Captures order information including order ID, user reference, embedded product list (with quantity and price), total amount, and payment reference.
- **Cart:** Maintains the items added by a specific user for future purchase, typically linked via user ID and storing an array of product references with quantities.
- **Payment:** Records transaction information such as payment ID, associated order ID, payment method (cash on delivery).

Order documents embed product summaries (product ID, quantity, and price) to allow fast reads without requiring joints.

User and order IDs are referenced across collections (e.g., in carts) to represent relationships.

Collections are schema-flexible, allowing easy modification and extension of entity attributes.

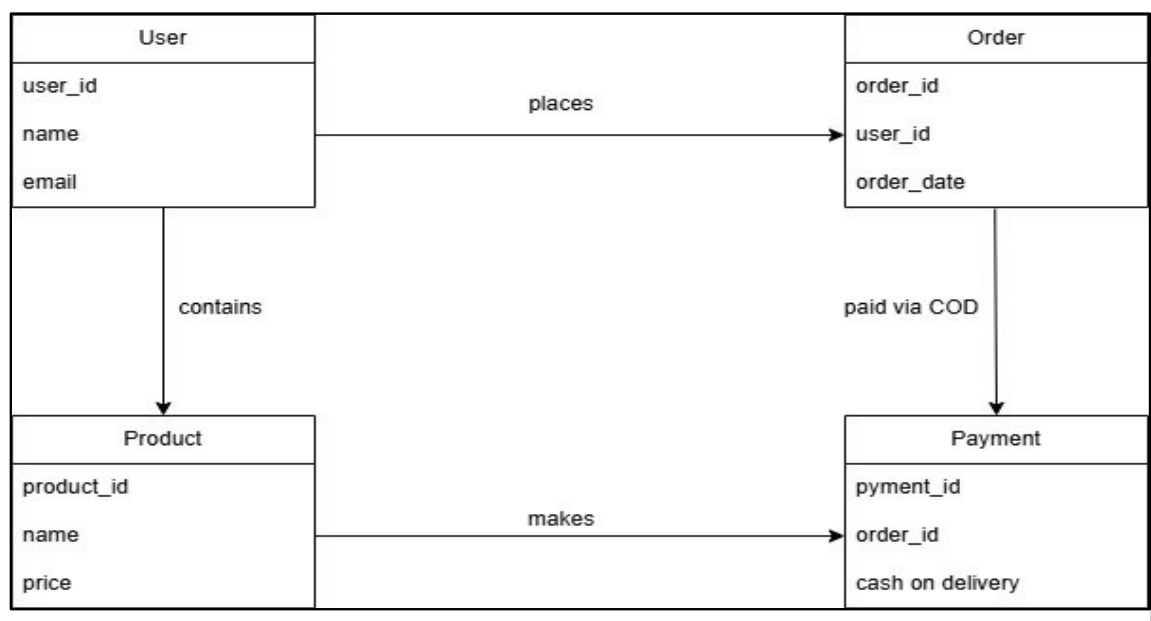


Figure 6.2 Entity Relationship Diagram (ERD) for the PRISVERDA

6.3 Process Flow/Representation

The process flow in Prisverda follows a logical sequence from user interaction to backend processing and data persistence, ensuring a smooth and secure shopping experience. The journey begins when a user accesses the platform and either registers or logs in using the authentication system. Upon successful login, session data is stored, and the user gains access to features like browsing products, adding items to the cart, and checking out. When a product is selected, its data is fetched from the MongoDB database and rendered dynamically through the React frontend. Cart actions—such as adding, removing, or updating quantities—are reflected in both the user interface and the `Cart` collection in the database. At checkout, the system retrieves the current cart items, calculates the total amount, and generates an order record in the `Orders` collection, embedding product and user details while marking the payment mode as Cash on Delivery (COD). Simultaneously, the stock levels in the `Products` collection are updated to reflect the purchase. The cart is then cleared, and an order confirmation is displayed to the user. For administrators, the process flow includes logging into a protected admin dashboard, managing products, viewing and updating order statuses, and controlling user access if necessary. This end-to-end data flow ensures that each module—authentication, product management, cart, and order processing—interacts consistently through RESTful APIs, with all operations securely managed on the backend and accurately reflected in the MongoDB database.

This process flow diagram depicts the step-by-step interaction from the user's perspective when placing an order on the platform:

- **Login/Register:** The user authenticates or creates a new account.
- **Browse Products:** The user views categories, filters, and selects items.
- **Add to Cart:** Selected items are added to the cart.
- **Checkout:** The user reviews the cart and proceeds to place an order.
- **Payment:** Cash on delivery.
- **Order Confirmation:** An order ID is generated, and the inventory is updated.

The backend services are triggered appropriately to manage database updates, payment, and product stock handling.

After a successful order, the cart is cleared, and the user is presented with a confirmation page displaying a unique order ID and details of the transaction. On the administrative side, authorized personnel can log into a protected admin dashboard to perform critical operations such as managing the product catalog, updating inventory, monitoring orders, modifying order statuses, and managing user access. Each of these interactions, whether initiated by the customer or the administrator, is handled through RESTful APIs, ensuring consistency, security, and scalability. This end-to-end flow not only guarantees robust e-commerce experience but also maintains data integrity and operational efficiency across the platform. The process flow diagram associated with this section illustrates these steps from a user's perspective, starting from login and ending with order confirmation, clearly highlighting the interconnected components that power the Prisverda platform.

This diagram shows the main steps users follow on the Prisverda clothing website, from browsing products to completing orders.

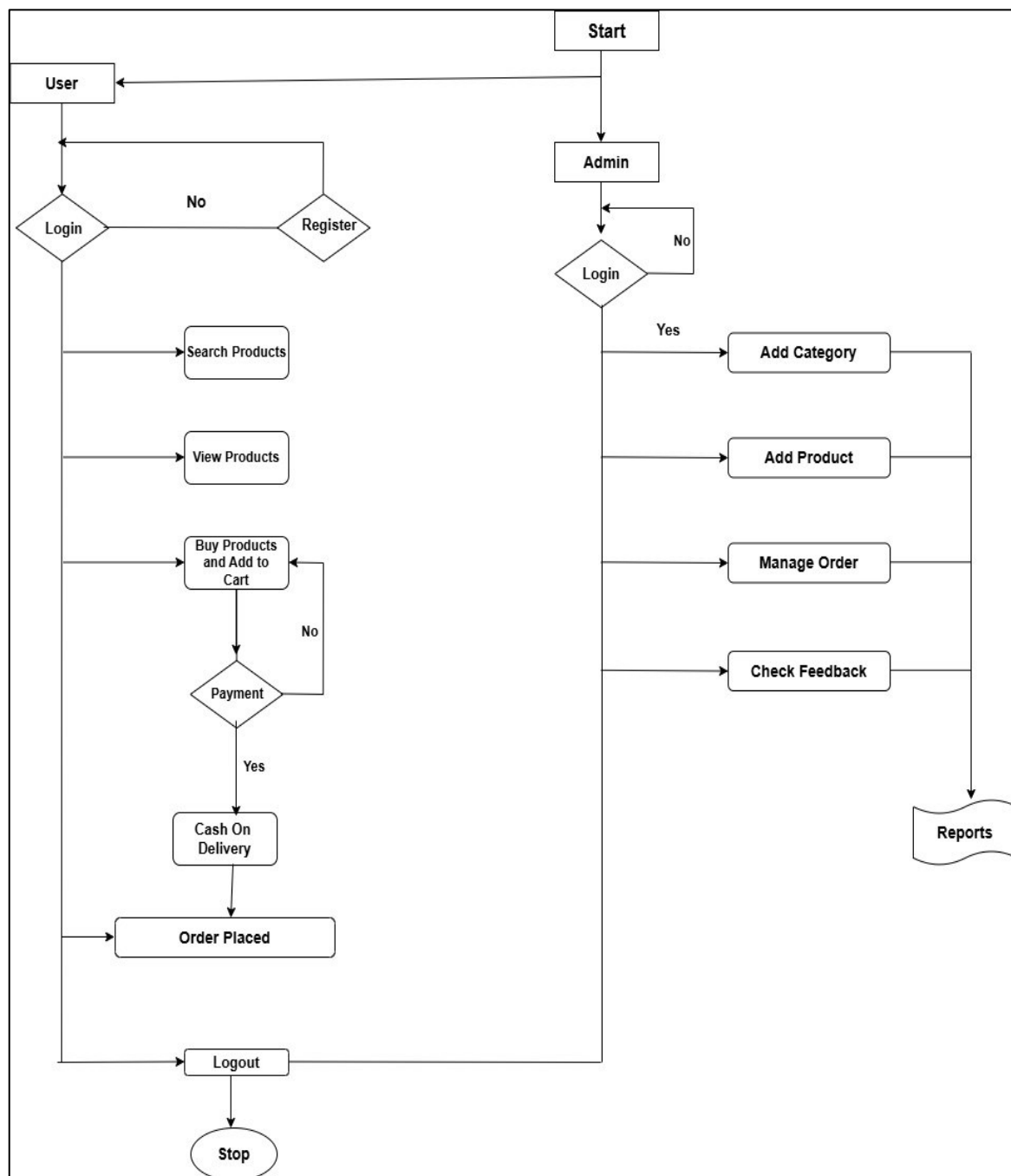


Figure 6.3 User Order Process Flow for the PRISVERDA clothing store

Chapter 7

Design Models

7. Design Models

System design serves as the **bridge between requirements and implementation**. It provides the architecture, models, and interfaces needed to guide how a system should function, interact, and behave under specific conditions. For Prisverda, thoughtful design ensures:

- Modular and scalable structure
- Reusable and responsive UI components
- Smooth user interaction flows
- Secure and optimized data handling

The system's design encompasses frontend layout, backend structure, database modeling, process flows, and security considerations.

7.1 Sequence Diagram

This sequence diagram shows the interaction between the frontend, user service, order service, inventory service, and payment (cash on delivery) during the order placement process.

The flow includes:

- Request initiation from user interface
- Validation by the user service
- Order creation request to the order service
- Payment (cash on delivery)
- Inventory update to reflect new stock levels
- Final confirmation and notification to the user

This ensures that operations are executed in a controlled and logical sequence.

By delineating each step in a sequential manner, this diagram serves as a clear blueprint of how a simple user action cascades through multiple services to result in a complete and coherent transaction. It highlights the dependencies and the order in which services must interact to maintain a seamless and reliable order placement mechanism. This flow ensures data integrity, user satisfaction, and operational efficiency across the entire e-commerce platform.

This diagram represents the chronological order of message exchanges between various entities, such as the user, frontend interface, backend server, and database. It outlines how a request is processed from initiation to completion, ensuring clarity in the communication flow.

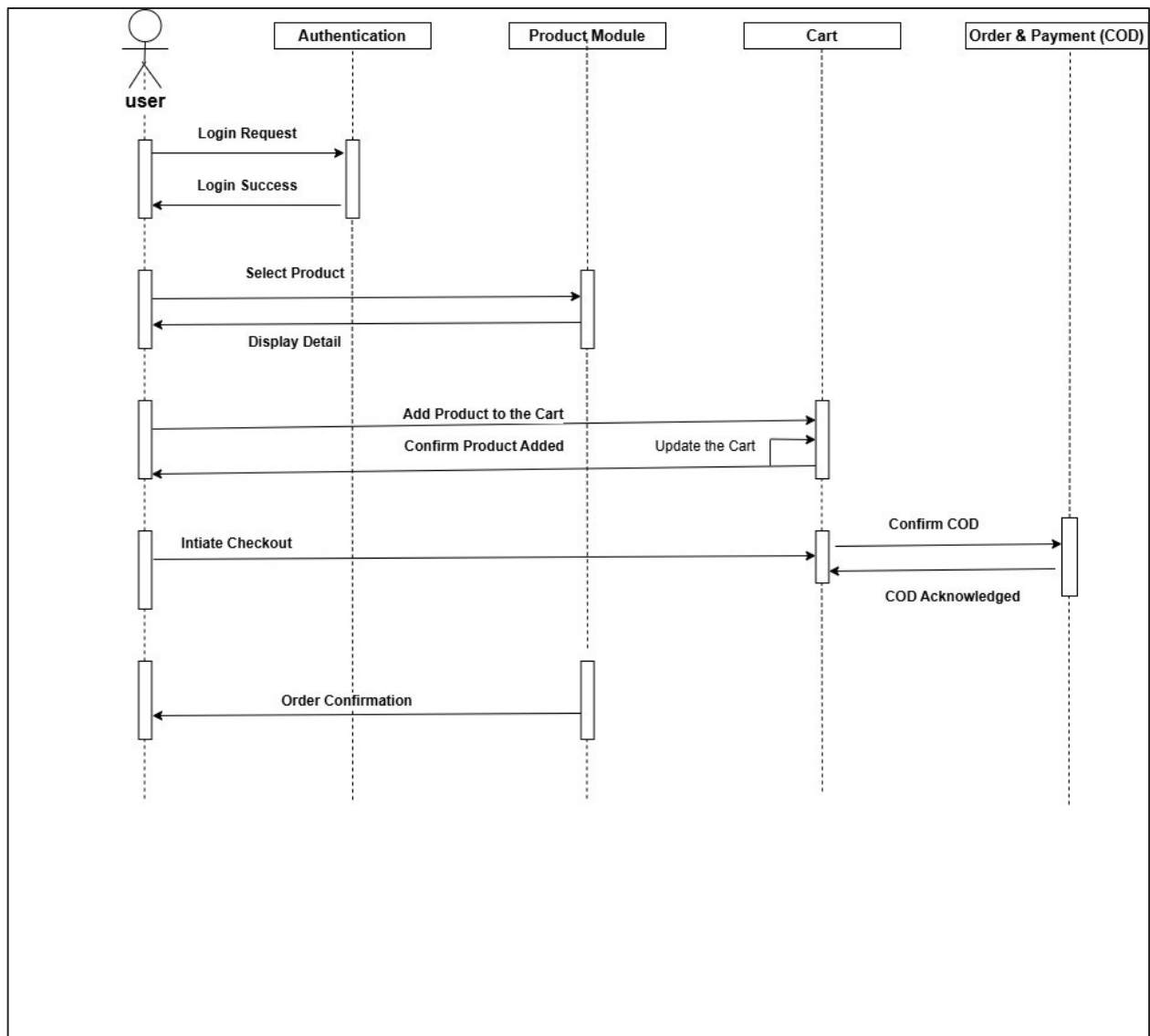


Figure 7.1 Sequence Diagram of PRISVERDA

7.2 Class Diagram

The class diagram for the Prisverda e-commerce system serves as a visual representation of the system's object-oriented design, showcasing the core classes, their attributes, and the relationships between them. This model helps in understanding how data is structured, manipulated, and managed within the system. Let's break down the key classes and their relationships:

7.2.1 User Class:

The User class represents the fundamental entity that interacts with the Prisverda e-commerce platform, encompassing both customers and administrators. It maintains essential user information, such as the user ID, name, email, password, and address, while also categorizing users based on roles like customer or admin. This class is equipped with methods for registration (`register()`), authentication (`login()`), and profile management (`updateProfile()`). By securely

storing and encrypting passwords, it ensures data privacy and access control. Additionally, the User class serves as the connecting point to other system modules, enabling users to manage orders, carts, and payment transactions. It also facilitates personalized user experiences by associating each user with specific orders, carts, and account settings.

- **Purpose:** Represents the users interacting with the system, such as customers and administrators.
- **Attributes:**
 - userID: Unique identifier for each user.
 - name: User's full name.
 - email: Email address for communication and login.
 - password: Encrypted password for authentication.
 - role: Defines user roles (e.g., customer, admin).
 - address: Shipping address for order delivery.
- **Methods:**
 - register(): Allows new users to register on the platform.
 - login(): Authenticates user credentials.
 - updateProfile(): Enables users to update their information.

7.2.2 Product Class:

The Product class is designed to encapsulate all relevant data regarding items available in the Prisverda store, including product ID, name, description, price, and stock quantity. Each product instance represents a distinct item, and the class provides methods such as getDetails() for retrieving product information, updateStock() for adjusting inventory levels, and applyDiscount() for implementing promotional pricing. By managing product data centrally, this class enables efficient catalog management and ensures data consistency across the platform. It also serves as the basis for the Cart and Order classes, allowing users to select products for purchase and track stock availability in real time.

- **Purpose: Manages product information available in the store.**
- **Attributes:**
 - productID: Unique identifier for each product.
 - name: Product name (e.g., "Beige Basic Collar Shirt").
 - price: Product price.
 - description: Detailed product description.
 - stock: Quantity available in the inventory.
- **Methods:**
 - getDetails(): Fetches product details.
 - updateStock(): Updates stock quantity after purchase.

- `applyDiscount()`: Applies any ongoing discounts.

7.2.3 Order Class:

The Order class is responsible for creating, tracking, and managing user purchases throughout their lifecycle. Each order instance is uniquely identified by an `orderId` and includes information such as the `userId`, order date, list of products, total amount, and order status (e.g., pending, shipped, delivered). The class provides methods like `placeOrder()` for initiating a purchase, `cancelOrder()` for reversing a transaction, and `trackOrder()` for monitoring order progress. The Order class also interacts closely with the Payment class to confirm payment processing and update order status. Additionally, it maintains a record of order history, allowing users to review past purchases and track ongoing deliveries.

• **Purpose:** Represents customer orders, linking users to their purchases.

• **Attributes:**

- `orderId`: Unique identifier for each order.
- `userId`: The user who placed the order.
- `orderDate`: Date and time of order placement.
- `orderStatus`: Current status (e.g., pending, shipped, delivered).
- `totalAmount`: Total cost of the order.

• **Methods:**

- `placeOrder()`: Creates a new order instance.
- `cancelOrder()`: Allows users to cancel an order.
- `trackOrder()`: Provides tracking information for a placed order.

7.2.4 Cart Class:

The Cart class acts as a temporary storage unit for products selected by users before proceeding to check out. It is associated with a specific user via the `userId` attribute and contains a list of products with their respective quantities and pricing. The class includes methods such as `addItem()` to add products, `removeItem()` to delete products, and `calculateTotal()` to compute the cumulative price of items in the cart. It maintains product availability checks to prevent over ordering and updates the total price dynamically as items are added or removed. Additionally, the Cart class is crucial in transferring selected products to the Order class for purchase processing and payment.

• **Purpose:** Temporarily stores selected products before purchase.

• **Attributes:**

- `cartID`: Unique identifier for each cart.
- `userId`: Reference to the user associated with the cart.

- Products : List of products added to the cart.
- totalPrice: Cumulative cost of items in the cart.

• **Methods:**

- addItem(): Adds a product to the cart.
- removeItem(): Removes a product from the cart.
- calculateTotal(): Updates the total price based on added items.

7.2.5 Payment Class:

The Payment class manages the financial transactions associated with user orders, ensuring that payments are securely processed and recorded. It includes attributes such as paymentID, orderID, paymentMethod (e.g., cash on delivery), amount, and paymentStatus. Key methods include processPayment() to handle transaction processing, confirmPayment() to verify successful payments, and refundPayment() to handle returns and cancellations. The class works in tandem with the Order class to confirm that the order total matches the payment amount and updates the order status upon successful payment. Additionally, it maintains transaction records, enabling users and administrators to track financial activities within the system.

• **Purpose: Manages payment processing for completed orders.**

• **Attributes:**

- paymentID: Unique identifier for each payment.
- orderID: The associated order for the payment.
- paymentMethod: Mode of payment (e.g., cash on delivery).
- amount: Payment amount.
- paymentStatus: Status of the payment (e.g., pending, completed).

• **Methods:**

- processPayment(): Processes the payment transaction.
- confirmPayment(): Confirms successful payment.
- refundPayment(): Handles refunds for canceled orders

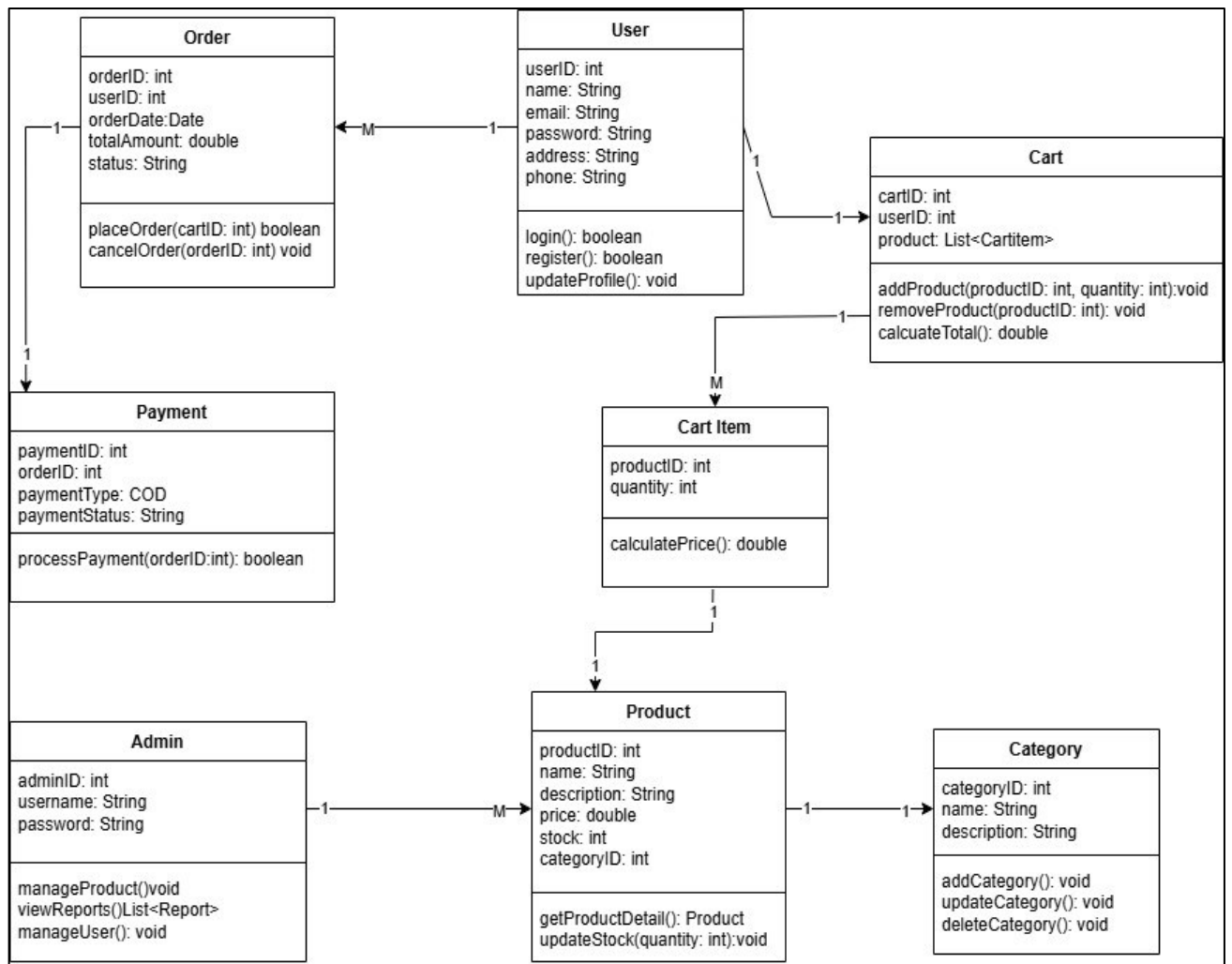


Figure 7.2 Class Diagram – System Structure

The class diagram provides a high-level view of the system’s structure by representing its classes, attributes, methods, and relationships. It outlines how different objects in the system are interconnected, helping in the object-oriented design of the application.

Chapter 8

Implementation

8. Implementation

Implementation is the stage where the system's functional components are developed and integrated into a cohesive application. Prisverda's implementation followed a modular and incremental development approach, using the MERN stack. The frontend was built with React.js, structured into components that represent UI elements such as product cards, navigation bars, and user forms. MaterialUI and Bootstrap libraries were used to ensure a responsive and aesthetically pleasing design. The application uses React Router for navigation and Context API for managing global state, such as user sessions and cart data.

The backend was built using Node.js and Express.js. It consists of RESTful API endpoints for handling authentication, user management, product operations, cart updates, and order processing. Middleware functions enforce authentication and error handling, while MongoDB, accessed via Mongoose, stores application data in a schema-based structure. Key entities such as users, products, orders, and carts are stored in separate collections, each defined by Mongoose models.

Authentication is handled through password hashing with bcrypt and, optionally, JSON Web Tokens (JWT) for protected routes. Product and user data are accessed via secure, paginated endpoints. The cart functionality tracks items per user and calculates totals, while order placement triggers inventory updates and saves an order snapshot. External services like Google Maps API were integrated for store location display, and deployment scripts were prepared for platforms like Heroku and Netlify. Implementation also followed best practices such as Git-based version control, modular code organization, and use of environment variables for secure configuration.

8.1 Algorithm

The core functionalities of the project are built on several algorithms that drive key modules. Below is the pseudocode or natural language explanation of how the main features of the platform operate:

1. User Authentication Algorithm:

Start

- The user enters login credentials (username and password).
- Validate credentials against the database.
- If credentials are correct, grant access to the user dashboard.
- If incorrect, display an error message and prompt for re-entry.
- **End**

2. Product Search Algorithm:

Start

- The user enters a search term in the search bar.
- The system queries the database for product matches.
- Display matching products with relevant details (name, price, image).
- Sort the products based on the user's filter (e.g., price, alphabetically).
- **End**

3. Order Processing Algorithm: •

Start

- The customer adds products to the shopping cart.
- Calculate total price including taxes.
- The user enters his/her details for delivery.
- Confirms the order.
- **End**

8.2 External APIs

The project integrates several external APIs to enhance functionality. These APIs handle tasks like payment processing, and location-based shipping. The table below provides details of each API used in the system:

Table 8.1 Details of APIs used in the project

Feature/Module	Description	Purpose of Usage	Functions/Classes Used
Authentication Module	Handles user login, sign-up, and logout functionalities	To allow users to register and securely log into their accounts	signUpUser(), loginUser(), logoutUser()
Product Management (Hardcoded)	Product data is hardcoded or updated manually via backend code	Admin updates products, prices, and stock levels manually	Product class, update functions in admin panel
Filtering System	Allows users to filter products alphabetically or by price	Enhances user experience in browsing and locating desired products	applyAlphabeticalFilter(), applyPriceFilter()

User Messaging Option	Provides users a way to send queries or messages to the admin/shop	To support communication between users and shop	sendMessageToAdmin(), receiveReply()
Order Management System	Stores and fetches order data from the website's database	To handle product order placements and status tracking	placeOrder(), getOrderHistory(), cancelOrder()
Shop Location (Static)	Shop's physical location is static and manually shown in contact/about sections	To inform users about the shop's physical presence	Static map/image or shopLocationDetails()

8.3 User Interface

The user interface (UI) is designed to provide a seamless shopping experience for customers. Key features include:

- A responsive design that adapts to various screen sizes and devices.
- An intuitive layout with categories, filters, and a search bar for easy navigation.
- Clear and concise designs ensure that both customers and administrators can interact with the platform effortlessly.

8.4 Home Page

The home page of the Prisverda e-commerce website is designed as the primary gateway for users, offering a clean, modern, and visually engaging layout that immediately captures attention. Dominated by a high-resolution banner featuring a model showcasing the latest fashion collection, the page promotes a sense of brand quality and style. The banner includes a prominent call-to-action button labeled **“Shop Now”**, which directs users to the product listing page, encouraging instant exploration and engagement. Positioned above the banner is a wellstructured navigation bar containing clearly labeled links to essential sections of the website, including **Home, Products, Collections, About, and Contact**. It also features icons for **search, wishlist, shopping cart**, and user account access, making it easy for users to find what they need with minimal effort. The top-right corner reflects the logged-in user's name, enhancing personalization. The homepage serves not only as an introduction to the platform but also highlights special promotions and handpicked products, reinforcing the brand's emphasis on **premium quality and user satisfaction**. Overall, the design ensures a seamless user experience while guiding users toward key areas of the site through intuitive navigation and compelling visuals.

The overall design of the homepage is driven by UX/UI best practices—clean typography, consistent branding, intuitive layout, and mobile responsiveness. Whether accessed on desktop,

tablet, or mobile devices, the interface adjusts fluidly to deliver a seamless and satisfying user experience.

Ultimately, the Prisverda homepage is more than just a landing space—it acts as a dynamic storefront, content hub, and navigational anchor. It successfully balances aesthetics with performance, guiding users effortlessly from discovery to action, while reinforcing the platform’s commitment to quality, style, and customer-centric design.

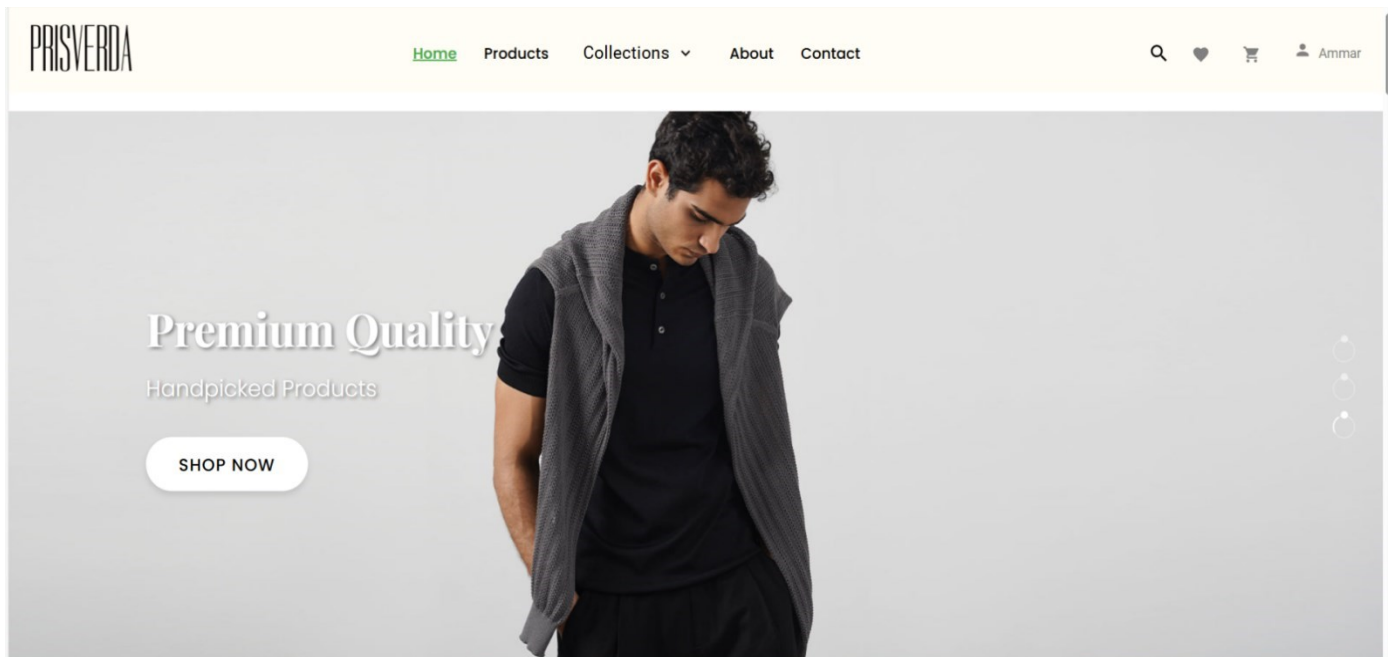


Figure 8.1 Home Screen of Prisverda

8.5 Services Page Interface design

The **"Our Core Values"** section on the Prisverda website serves as a visual and textual representation of the brand's foundational principles and service commitments. Displayed in a clean, minimalistic layout, this section showcases five distinct pillars that define Prisverda’s operational philosophy. Each value is presented in its own card-style box with bold headings and concise supporting text, ensuring clarity and readability. The first card emphasizes **Fast Delivery**, promising customers quick and reliable shipping services worldwide, which highlights the brand’s dedication to convenience and efficiency. The second card, **Quality Guaranteed**, assures users of premium materials and expert craftsmanship, reinforcing the site’s commitment to product excellence. The third value, **Sustainable**, promotes eco-friendly practices and materials, signaling Prisverda’s awareness of and responsibility toward environmental sustainability. The fourth card, **Customer First**, declares the brand’s unwavering

dedication to customer satisfaction, putting user needs and experiences at the center of its operations. Finally, the **Recommendations** value introduces the concept of personalized product suggestions, aligning with modern e-commerce trends that emphasize customization and user-centric shopping. This section not only informs users of Prisverda’s service philosophy but also builds trust and transparency, creating a strong emotional connection with current and potential customers.

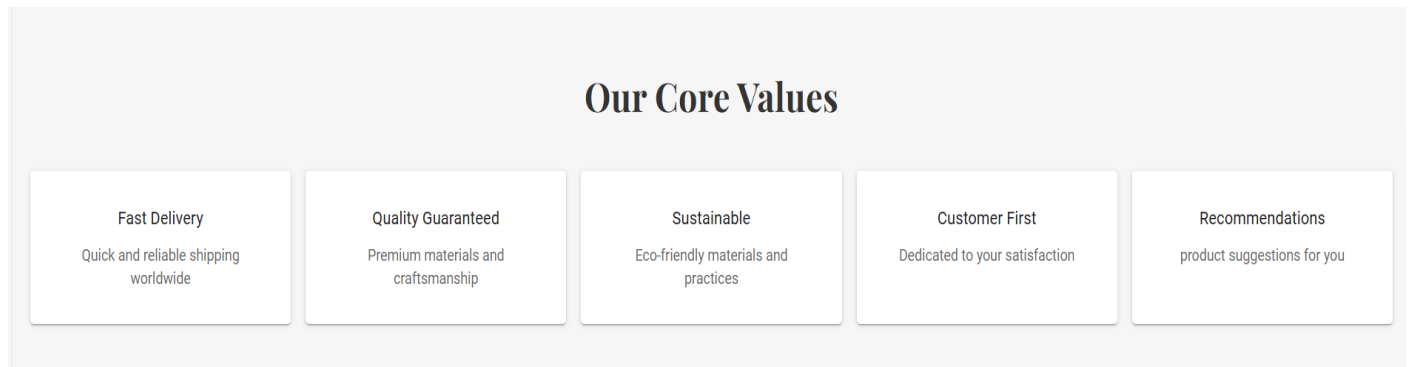


Figure 8.2 Services Page Interface design

8.6 Intuitive Product Categorization

The webpage presents an intuitive and visually engaging product categorization layout that enhances the browsing experience by organizing various apparel types such as T-shirts, shirts, and pants into distinct, easily navigable sections. Each category is represented through high-quality, thoughtfully composed images that not only showcase the aesthetic appeal of the products but also evoke the style and essence of each collection. Positioned prominently at the top of the section is the title “Our Collections,” which serves as a clear and inviting focal point, guiding users to explore the different offerings. Below each category image, bold and capitalized labels distinctly identify the type of clothing, ensuring instant recognition and reducing cognitive load for users. Accompanying each label is a visually appealing “Shop Now” call-to-action button, designed to encourage immediate engagement by seamlessly directing users to the corresponding product listings. The layout embraces a clean, minimalist aesthetic that prioritizes product visibility while ensuring the navigation bar, logo, and other branding elements remain subtly in the background. This thoughtful balance between visual clarity and interactive design not only supports ease of use but also contributes to a polished, professional, and immersive user experience.

Our Collections

T-SHIRTS

SHOP NOW

SHIRTS

SHOP NOW

PANTS

SHOP NOW

Figure 8.3 Intuitive Product Categorization of Prisverda

8.7 Detailed Item Display Interface

The Detailed Item Display Interface on the Prisverda e-commerce platform is meticulously crafted to offer users an immersive and seamless shopping experience. At the forefront of each product page is a high-resolution image gallery that allows customers to view the item from multiple angles, including zoom and carousel features, providing an in-depth visual understanding of the product. The product name such as “Beige Basic Collar Shirt” is presented in bold, prominent typography, ensuring it captures immediate attention, while the price is clearly displayed just beneath, making it easy for users to assess affordability immediately. A concise yet informative description follows, outlining key product features such as material, design, and fitness, aimed at helping users make confident purchasing decisions. To streamline the buying process, the interface includes a size selector, a quantity input, and a visually prominent “Add to Cart” button with responsive feedback upon interaction. Additionally, a heart icon allows users to save the product to a wishlist for future reference. Further enriching the user experience, a “You Might Also Like” section showcases related items based on browsing behavior or category, encouraging deeper site engagement and potentially increasing overall order value. Designed with responsiveness in mind, the interface maintains its functionality and clarity across all devices, supporting accessibility features like screen reader compatibility and keyboard navigation. The layout is clean and intuitive, with optional sections for customer reviews, shipping details, return policies, and size guides, all neatly organized under collapsible tabs or secondary panels. Overall, this interface not only prioritizes visual appeal and usability but also strategically guides users through the path to purchase with minimal friction.

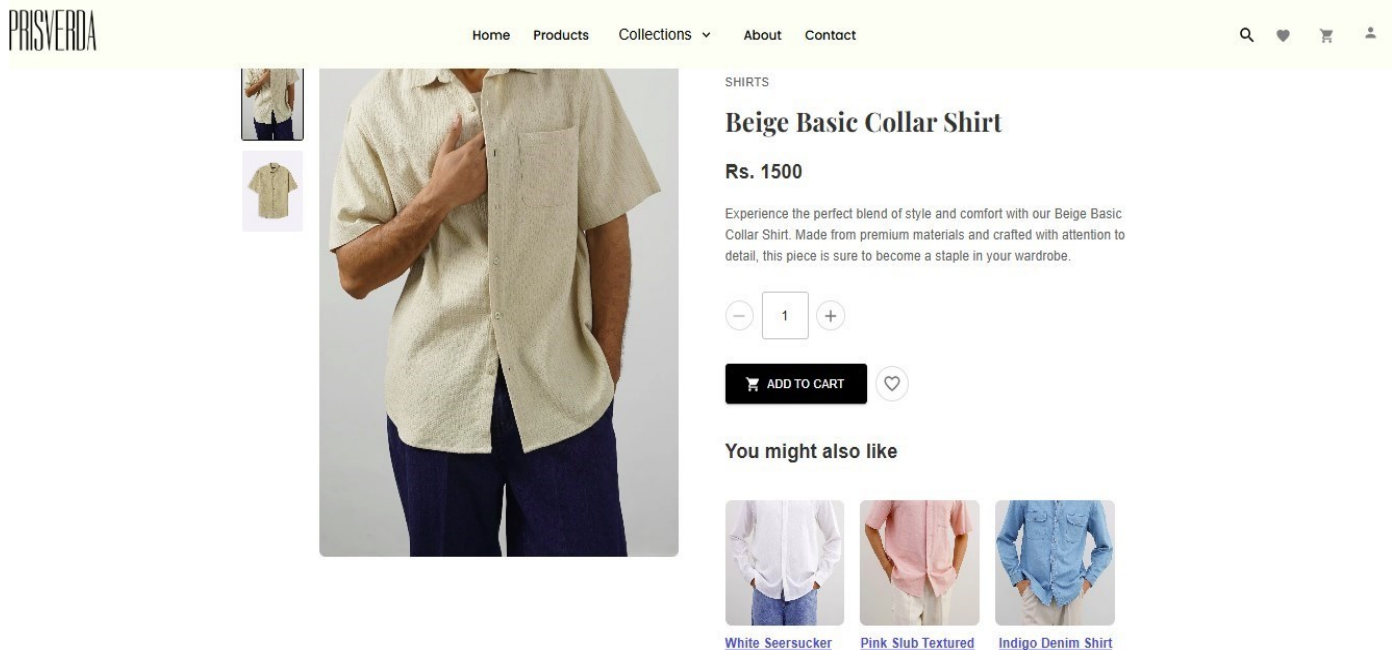


Figure 8.4 Detailed Item Display Interface of Prisverda item

8.8 Highlighted Features

The "You Might Also Like" section on the Prisverda webpage is a thoughtfully designed feature that plays a crucial role in enhancing user engagement and driving additional sales. Strategically positioned directly beneath the main product display area, this section serves as a subtle yet effective recommendation tool, presenting users with a curated selection of related products. Each item within this grid is showcased with high-quality, consistent product imagery that aligns with the overall visual style of the website, accompanied by clearly displayed product names and prices. The design ensures a smooth and uninterrupted browsing experience by maintaining uniformity in layout, spacing, and typography, which allows users to shift their attention effortlessly from one product to another. By featuring a variety of shirts and other items in different colors, patterns, and styles, Prisverda emphasizes its commitment to diversity in fashion offerings, catering to a wide range of customer preferences. Moreover, this section reinforces Prisverda's brand promise of delivering stylish, high-quality clothing at affordable prices. Beyond enhancing the overall aesthetic appeal of the page, the "You Might Also Like" feature is strategically engineered to encourage deeper exploration of the product catalog, thereby increasing the likelihood of **cross-selling** complementary items and **upselling** more premium alternatives. As a result, this section not only adds functional value to the shopping experience but also supports the brand's broader goals of boosting customer retention and maximizing average order value.

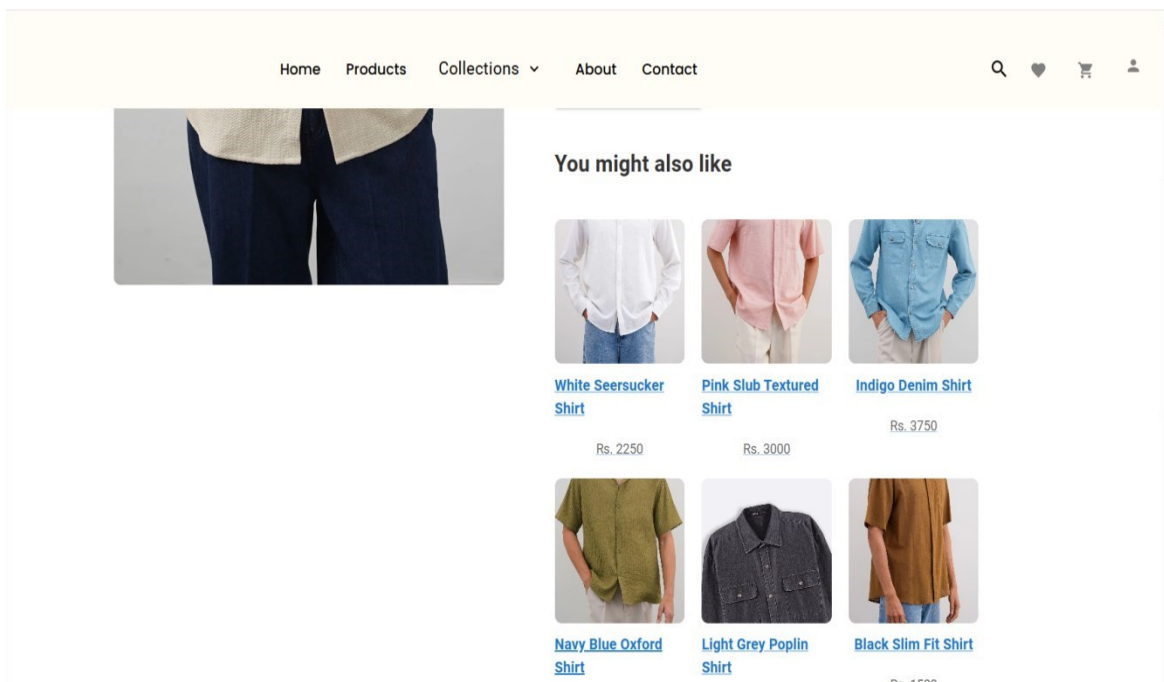


Figure 8.5 Highlighted Features Interface of Prisverda

8.9 Footer Section of the Website

The footer section of the Prisverda website is thoughtfully designed to provide essential navigation links while reinforcing the brand's identity and customer support services. It is divided into well-organized sections, ensuring easy access to key information.

On the left, the Prisverda section includes links to pages such as About Us, Conditions, Our Journals, Careers, Affiliate Program, and Prisverda Press, providing visitors with comprehensive insight into the brand's background, policies, and opportunities.

The Customer Service section is prominently positioned in the center, listing vital links such as FAQ, Contact, Privacy Policy, Returns & Refunds, Cookie Guidelines, and Delivery Information, ensuring that customers can quickly locate assistance for any inquiries or concerns.

To the right, the Contact Us section offers direct communication channels, including an email address and a customer support phone number, emphasizing Prisverda's commitment to accessibility and customer satisfaction.

The footer also features a promotional tagline – "Drip 2024 – Step into 2024 with the freshest drip that defines your style—where comfort meets bold expression," reinforcing the brand's fashionable and trendy image.

Additionally, icons for social media platforms (such as Facebook, Twitter, and YouTube) are prominently displayed, inviting users to connect and stay updated on the latest products and promotions.

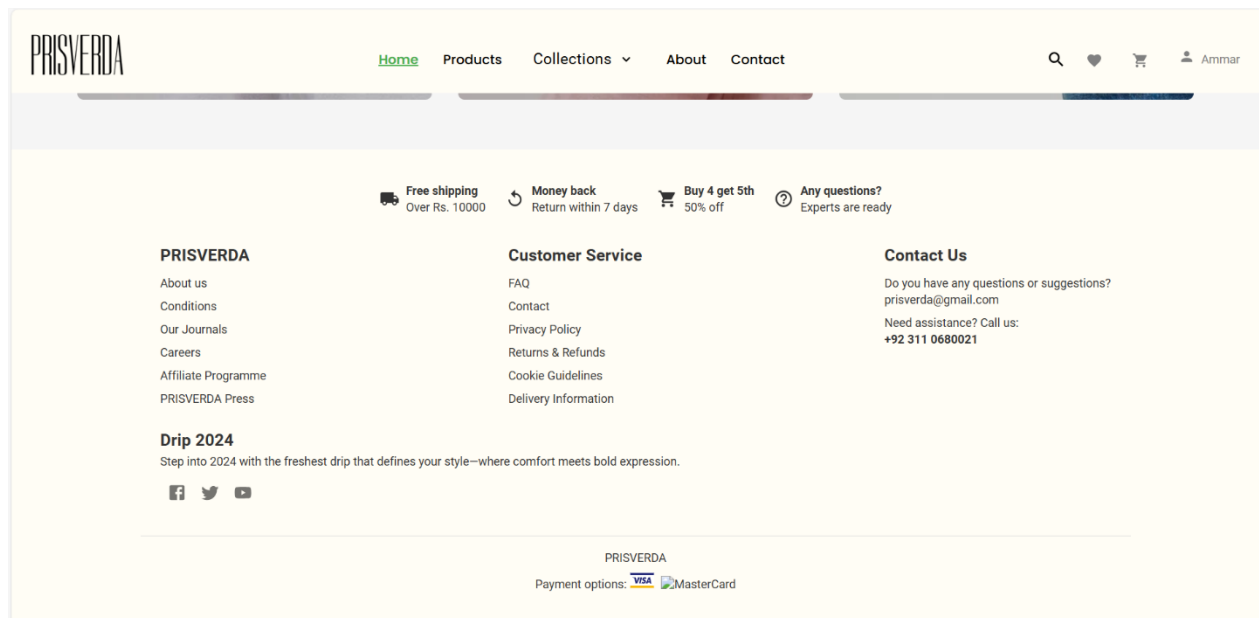


Figure 8.6 Footer Section of the Prisverda

8.10 Contact Us Page

The Contact Us page on the Prisverda website is thoughtfully designed to provide a seamless communication channel for users seeking to submit inquiries, provide feedback, or request support.

At the top of the page, a prominent header image featuring warm, welcoming tones is paired with bold, capitalized text that clearly states, "CONTACT US," setting a welcoming yet professional tone. The visual design reinforces the brand's commitment to user engagement and accessibility.

Below the header, the contact form is strategically positioned to capture user information effectively. The form is simple and easy to navigate, consisting of clearly labeled input fields for essential details:

Name: Allows users to enter their full name, ensuring the brand can address them appropriately.

Email: Captures the user's email address, serving as the primary method of response and followup.

Message: A larger text field for users to provide detailed information regarding their inquiries, suggestions, or feedback.

A clearly defined submit button is placed beneath the input fields, enabling users to send their messages effortlessly. The button is designed with a contrasting color to draw immediate attention and encourage action.

Adjacent to the contact form, a Google Maps widget is embedded to display the physical location of Prisverda's business premises. This interactive map not only offers directions but also reinforces the brand's transparency and credibility. The map is accompanied by contact details, including the business address and phone number, ensuring users can connect through multiple channels.

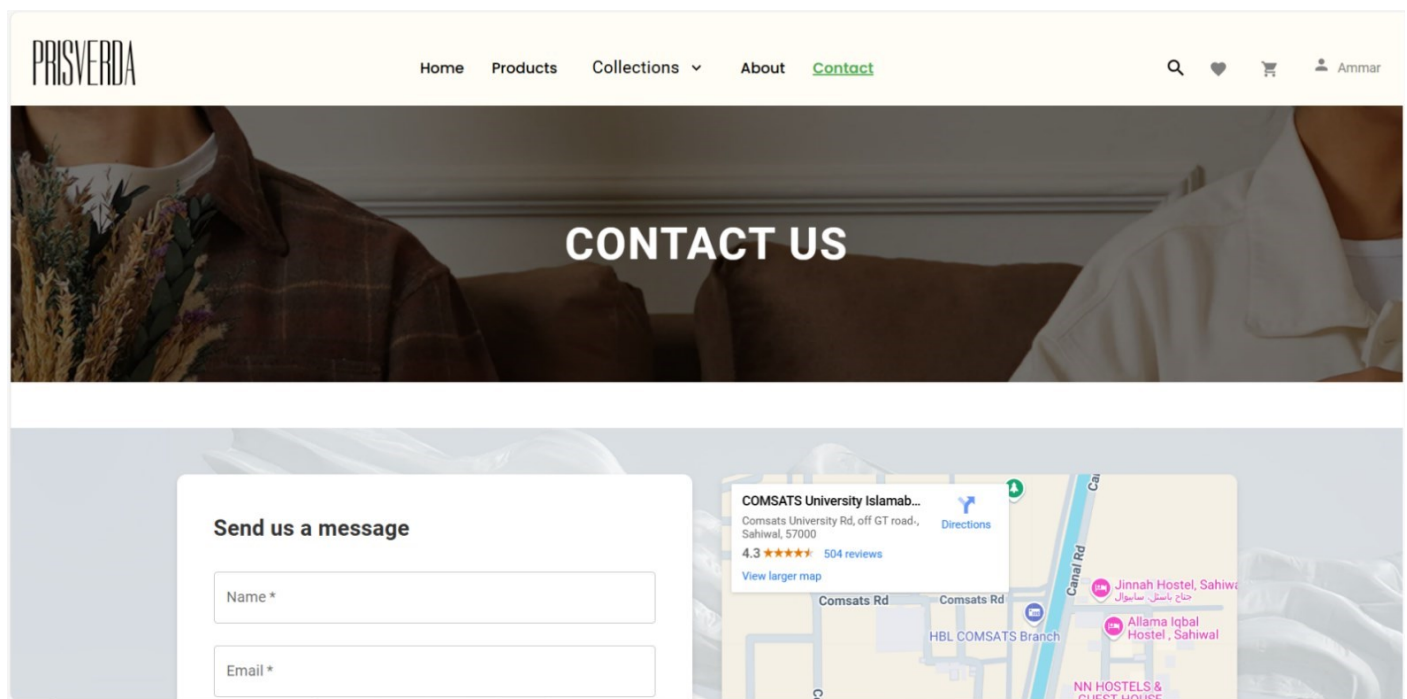


Figure 8.7 Contact Us Page Interface of Prisverda

8.11 User Login

The User Login (Sign In) interface serves as a secure entry point for users to access their personal accounts within the application. It is designed with a clean and user-friendly layout, featuring two essential input fields, one for the user's email address and another for their password. The email field requires a properly formatted address, ensuring that each user is uniquely identified in the system. The password field is masked by default to enhance security and prevent unauthorized onlookers from viewing the typed characters. To improve usability, a

visibility toggle icon (typically an eye icon) is included, allowing users to reveal or hide their password as needed for accuracy. Below the input fields is a prominent "Sign In" button, which, upon being clicked, submits the entered credentials to the backend for authentication. If the credentials match a registered user, access is granted; otherwise, an error message may be shown. Additionally, the interface includes a subtle prompt for new users "Don't have an account? Sign up" which provides a direct link to the registration page, making it easy for first-time users to create an account. This login page is typically secured with HTTPS and may also be integrated with features like session management, CAPTCHA, or two-factor authentication (2FA) for enhanced security.

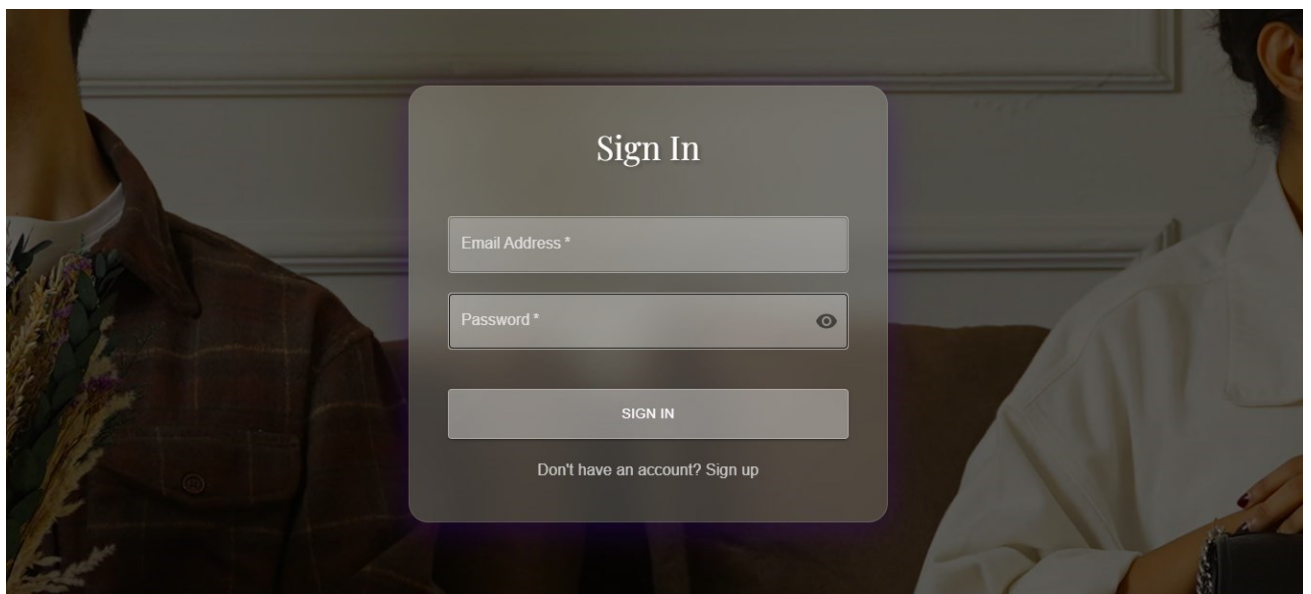


Figure 8.8 User Login Interface of Prisverda

Chapter 9

Testing and Evaluation

9. Testing and Evaluation

This chapter presents the comprehensive testing and evaluation phase of the e-commerce website project, a critical step in the software development lifecycle. The purpose of this phase is to ensure that the platform is **functionally correct, reliable, secure, and user-friendly** before it is made available to end-users. Testing was conducted using both **manual and automated approaches**, covering a range of scenarios from simple user interactions to complex backend operations.

Manual testing played a crucial role in identifying **visual issues, navigation problems, and responsiveness across various devices and browsers**. This included testing the website on desktop, tablet, and mobile devices using browsers like Chrome, Firefox, and Safari to ensure consistent behavior. Testers manually interacted with all major features such as user registration and login, product browsing, cart management, and order placement to verify that each function worked as expected and provided a smooth user experience. Edge cases, like attempting to checkout with an empty cart or logging in with incorrect credentials, were tested to validate that proper error handling was in place.

In parallel, unit testing was applied to individual components and functions. On the frontend, this involved checking the behavior of UI components such as the login form, product cards, and cart display. These tests confirmed that components rendered correctly, updated their state based on user input, and triggered appropriate actions. On the backend, unit testing focused on validating functions such as price calculations, password hashing, and session verification.

Functional testing evaluated complete user workflows to ensure that they followed the intended paths and produced correct results. For example, placing an order through the Cash on Delivery (COD) system involved testing product selection, cart updates, address confirmation, and final order creation. The entire flow was tested to ensure that product stock was accurately updated in the database and that the order was recorded with all relevant details.

Integration testing verified the interaction between different modules of the application. For instance, when a product was added to the cart, the system confirmed that this update was reflected in both the frontend UI and the backend database. Similarly, successful user authentication had to correctly influence session management and unlock protected routes such as the checkout page or admin panel.

While manual testing was the primary method due to its flexibility and visual feedback, basic automated testing was also implemented using tools like Jest and Postman. Jest was used to run unit tests on backend logic such as verifying API responses and ensuring calculations returned expected values while Postman was used to test the integrity of RESTful API endpoints. These tools allowed

repeatable tests to be written and executed quickly, particularly useful during each sprint when new features were added or old ones modified.

Ultimately, this rigorous testing phase helped identify and fix bugs, improve user interface consistency, and confirm that all system features aligned with both functional requirements and user expectations. The result was a stable, secure, and fully functional e-commerce website ready for deployment and real-world use.

9.1 Manual Testing

Manual testing was conducted as a core quality assurance activity to verify that each feature of the Prisverda e-commerce website functioned as expected through direct, hands-on interaction with the system. This approach involved executing predefined test cases without the assistance of automated testing tools, allowing testers to assess the user interface, workflows, responsiveness, and functional correctness from the perspective of an actual end user.

The goal of manual testing was to ensure that all core features such as user registration and login, product browsing, cart operations, checkout, and order placement operated smoothly, accurately, and intuitively. Testers explored each module under various usage scenarios and provided real-time feedback on usability issues, visual inconsistencies, navigation problems, and logical errors.

During the testing phase, different user roles (customer and admin) were simulated to validate role-based access control. For example, customers were prevented from accessing admin pages, and admins could view and manage all product and order data securely. Each form, such as login, registration, and checkout, was tested with both valid and invalid inputs to ensure proper error messaging and input validation.

Testers also paid close attention to edge cases and unexpected behaviors, such as:

Attempting to place an order with an empty cart

Trying to register using an email that already exists

Logging in with incorrect credentials

Adding out-of-stock products to the cart

Submitting forms without filling mandatory fields

9.2 System testing

System testing was conducted after the completion of the core development phase to validate that all individual modules and integrated features of the Prisverda e-commerce website functioned cohesively as a complete and reliable system. This phase focused on verifying the system's compliance with the functional and non-functional requirements defined during the requirements analysis stage.

The objective of system testing was to evaluate the end-to-end behavior of the platform under real-world usage scenarios and to ensure that each component interacted correctly with others. This included testing key user-facing features such as user registration, login, password management, profile updates, product browsing, filtering, cart management, checkout using Cash on Delivery (COD), and order confirmation. For administrative users, it included verifying access control, product management capabilities, and order monitoring.

9.3 Unit Testing

9.3.1 Unit Testing 1: User Login

Testing Objective: To verify that the login form functions correctly.

Table 9.1 To verify that the login form functions correctly

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Attempt to login with valid credentials	Username: user001Password: pass123	Successfully logged in and redirected to the homepage	Pass
2	Attempt login with invalid password	Username: user001Password: wrong pass	Error message: "Invalid username" displayed	Pass

9.3.2 Unit Testing 2: Edit Profile

Testing Objective: To ensure that the edit profile form works correctly.

Table 9.2 ensure that the edit profile form works correctly.

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Update user email	New Email: prisverda@gmail.com	Profile updated successfully	Pass

2	Leave required	Name field left blank	Error message: “Fill	Pass
	fields empty and submit		the fields” displayed	

9.4 Functional Testing

Functional testing was carried out after unit testing to ensure that all modules worked according to their specifications. Each function was tested in isolation with valid and invalid inputs to determine correct behavior.

9.4.1 Functional Testing 1: Login with Different Roles

Objective: To confirm that users are redirected to the correct dashboard based on their role.

Table 9.3 To confirm that users are redirected to the correct dashboard

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Login as customer	Username: user001 Password: pass123	Redirected to customer homepage with product listings	Pass

9.5 Integration Testing

Integration testing was performed to check how different modules interact with one another and whether data flows smoothly across modules.

Table 9.4 Integration Testing Results for Core Functionalities

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Login and signup	Valid email + new email	Successfully redirected and account registered	Pass
2	Add product to cart and proceed to checkout	Select product + click checkout	Product appears in checkout page with correct pricing	Pass
3	Search for a product and apply filters	Product keyword + price filter	Filtered results displayed accurately	Pass

9.6 Automated Testing

Automated testing was performed on selected functionalities using a simple tool like **Selenium** to ensure consistency and speed in repetitive tasks such as login and navigation checks.

9.7 Tools Used

The following table summarizes the automated testing process, listing the tools used, their descriptions, and the corresponding test cases or features they were applied to.

Table 9.5 Automated Testing Results Using Selected Tools

Tool Name	Tool Description	Applied On	Results
Selenium	Open-source tool for automating web browsers	Login page, Search feature, Navigation links	Test scripts executed successfully; all results passed

Chapter 10

Conclusion and Future work

10. Conclusion and Future Work

The e-commerce website was successfully implemented and tested using both manual and automated methods. Testing ensured that the website's core functionalities, such as login, product listing, and user account, were fully operational. In the future, more advanced features such as payment gateway integration, email confirmation, user reviews, and product recommendations can be implemented to enhance the website's functionality and user experience.

10.1 Conclusion

In conclusion, the **Prisverda e-commerce website** was successfully designed, developed, and tested as a comprehensive, user-friendly platform with the goal of delivering a seamless and satisfying online shopping experience. Developed using the MERN stack, Prisverda effectively integrates a responsive frontend interface with a secure and scalable backend, resulting in a complete full-stack solution. The platform caters to a diverse audience, particularly within the modest fashion segment, by offering a wide range of curated products designed to meet the preferences and expectations of both casual and style-conscious users.

The implementation of core functionalities such as **user registration and login, product browsing and filtering, shopping cart management, and order placement via Cash on Delivery (COD)** significantly enhances customer engagement and usability. These features allow users to interact with the system in a natural and intuitive way from discovering products and adding them to the cart, to completing an order with minimal friction. For first-time buyers and users in markets with limited trust in digital payments, the COD option provides an added layer of confidence, encouraging transaction completion.

The **user interface (UI)** has been carefully designed to be both visually appealing and highly functional. Key design elements include high-resolution product imagery, clear typography, consistent color schemes, and intuitive navigation menus. These elements work together to deliver a professional shopping environment that closely aligns with modern web design standards. The UI is also fully responsive, ensuring that the website performs optimally across various screen sizes and devices, from desktop computers to smartphones.

On the administrative side, backend features allow authorized administrators to manage product listings, update stock quantities, and monitor user orders efficiently through a protected dashboard. This ensures that the site can remain dynamic and up to date, with new inventory and product changes

reflected in real time. By enabling real-time database updates and modular data handling, the backend supports scalability and future extensibility.

Overall, **Prisverda provides a strong technological and design foundation for a modern ecommerce solution**. It not only meets the primary objectives outlined at the beginning of development such as enhancing user trust, supporting local payment methods, and delivering responsive shopping experience but also establishes a framework that can be extended with advanced features in future iterations. With proper enhancements, such as digital payments, product recommendations, and mobile app integration, Prisverda has the potential to evolve from a student project into a commercially viable e-commerce platform that serves a wider market with professionalism and convenience.

10.2 Future Work

As Prisverda continues to mature beyond its current version, there are several key opportunities for enhancement that can significantly elevate the platform's functionality, user experience, and scalability. These future improvements span both **functional upgrades** and **intelligent automation features**, aimed at transforming Prisverda from a solid academic prototype into a full-featured, competitive e-commerce solution. Implementing these improvements would align the platform with current industry trends and user expectations, while also expanding its market reach.

10.2.1 Payment Gateway Integration

One of the most important next steps is the integration of a secure and reliable **payment gateway**, such as **Stripe**, **PayPal**, or local services like **JazzCash** or **EasyPaisa**. Currently, Prisverda supports only **Cash on Delivery (COD)**, which, while effective in regions with low trust in digital transactions, can limit customer options and operational efficiency. Introducing online payment options will:

- Offer flexibility for tech-savvy users who prefer card or wallet payments
- Improve transaction speed and reduce manual handling of cash
- Enhance trust through encryption and fraud protection
- Allow for future implementation of refunds, returns, and invoice generation

Integrating a payment gateway will also involve securing compliance with standards like **PCI-DSS**, protecting sensitive user and financial data.

10.2.2 Real-Time Order Tracking System

Adding a **real-time order tracking system** would provide users with ongoing visibility into their order status — from confirmation to shipment and delivery. This system could include:

- Status updates (e.g., “Order Confirmed”, “Packed”, “Out for Delivery”, “Delivered”)

- Estimated delivery timelines
- Live location tracking via courier APIs
- Email or SMS alerts for each stage of the process

This level of transparency builds user trust, reduces customer support inquiries, and enhances the overall reliability and professionalism of the platform.

10.2.3 Personalized Recommendation Engine

A major advancement in user experience would be the implementation of a **recommendation engine**. By analyzing user data such as browsing history, product views, purchase records, and even time spent on pages the system can suggest personalized products in real time. Techniques may include:

- **Collaborative filtering** (suggest items based on similar users' behavior)
- **Content-based filtering** (recommend similar products based on item features)
- **Hybrid models** for improved accuracy

This feature not only improves customer experience but also increases average order value, boosts product discovery, and drives higher conversion rates.

10.2.4 Email and SMS Notifications

Automated **email and SMS notifications** can enhance communication and retention by keeping users informed at every step of their shopping journey. Use cases include:

- Sending order confirmations and receipts
- Shipping updates and delivery alerts
- Password reset or account-related actions
- Promotional campaigns or flash sale alerts

Platforms such as **SendGrid**, **Mailchimp**, or **Twilio** can be integrated for efficient and secure message delivery. These notifications foster stronger customer engagement and can be tailored to user behavior for improved marketing impact.

10.2.5 Mobile Application Development

To meet the growing trend of mobile commerce, Prisverda can be expanded into a **dedicated mobile application** using frameworks like **React Native** or **Flutter**. A mobile app provides:

- Faster loading times and better performance than mobile browsers
- Push notifications for direct communication
- Offline browsing and catching features

- Access to device features (e.g., camera for QR codes, GPS for delivery)

Given that a significant portion of online shopping occurs on smartphones, having a native mobile app can greatly increase accessibility, convenience, and user retention.

10.2.6 Multilingual and Multi-Currency Support

To broaden its global reach, Prisverda should support **multiple languages and currencies**, allowing international users to browse and shop comfortably. Implementing internationalization (i18n) involves:

- Translating all UI text, buttons, and messages into supported languages (e.g., Urdu, Arabic, English)
- Allowing users to select their preferred language and region
- Displaying prices in local currencies with real-time exchange rates
- Formatting dates and units according to regional standards

This feature is essential for **global expansion** and enhances accessibility for diverse user groups.

Conclusion of Future Work

Implementing these forward-looking enhancements will significantly strengthen Prisverda's position as a modern, scalable, and user-centric e-commerce platform. Each feature addresses a specific aspect of the customer journey from smoother transactions and real-time updates to personalized recommendations and mobile accessibility. Moreover, features like multilingual support and payment integration lay the groundwork for **international growth**, while recommendation engines and notification systems align Prisverda with **next-generation e-commerce standards**.

By pursuing this roadmap, Prisverda will evolve from a functional web prototype into a competitive, full-scale online shopping solution capable of meeting the high expectations of today's digital consumers.

11. References

1. w3schools. (n.d.). Retrieved from HTML HOME:
<https://www.w3schools.com/html/default.asp>
2. Frontend. (n.d.). Retrieved from w3schools:
<https://www.w3schools.com/html/default.asp> html. Retrieved from w3schools:
<https://www.w3schools.com/html/default.asp> nodeJS. (n.d.). coursera. (n.d.).
Retrieved from <https://www.coursera.org/courses?query=database>
3. Visual studio. (n.d.). Retrieved from
<https://visualstudio.microsoft.com/vs/community/>
4. Visual studio. (n.d.). Retrieved from VS CODE:
<https://visualstudio.microsoft.com/vs/community/>
5. MongoDB. (n.d.). Retrieved from MongoDB Official Documentation:
<https://www.mongodb.com/docs/>
6. React.js. (n.d.). Retrieved from React.js Official Documentation:
<https://react.dev/>
7. Material-UI. (n.d.). Retrieved from Material-UI Documentation:
<https://mui.com/>
8. Adobe Illustrator. (n.d.). Retrieved from Adobe Illustrator Official Site:
<https://www.adobe.com/products/illustrator.html>
9. Figma. (n.d.). Retrieved from Figma Official Website: <https://www.figma.com/>
10. Google Maps API. (n.d.). Retrieved from Google Maps API Documentation:
<https://developers.google.com/maps>